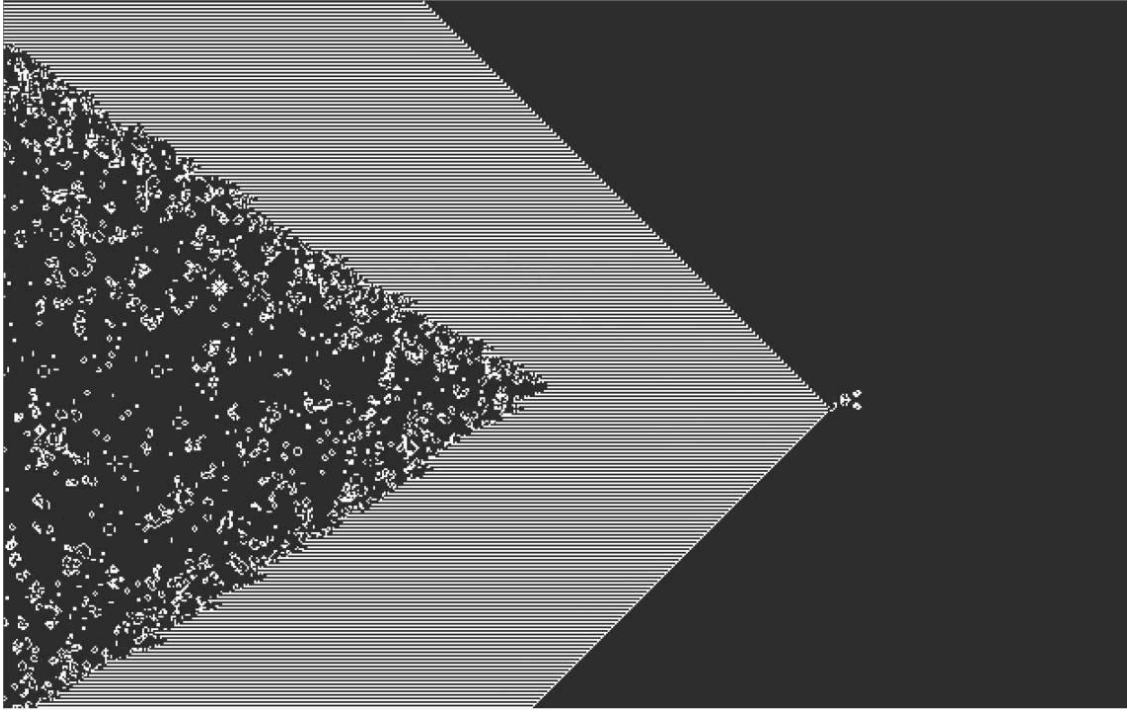


INGENIERÍA DE RENDIMIENTO: CONWAY'S – GAME OF LIFE



Índice

Índice.....	1
El juego de Conway 's game of life.....	2
Análisis de las características del algoritmo.....	3
Funcionamiento computacional.....	3
Estructuras de datos y disposición en memoria.....	3
Núcleo básico de cómputo.....	3
Patrón de computo.....	4
Vectorización.....	4
Complejidad algorítmica.....	5
Localidad temporal.....	5
Intensidad aritmética.....	5
Complejidad espacial.....	5
Herramienta propia de análisis del programa.....	6
Optimizaciones del programa y resultados parte 1.....	7
Cómo compilamos.....	7
Versión Optimizaciones directas.....	8
Cómo funciona.....	8
Resultados.....	8
Datos de rendimiento en tiempo (resumen, compilador versión reciente):.....	8
Datos globales del código directo (compilador versión reciente):.....	9
Limitación.....	10
Posibles vías de mejora.....	13
Versión Optimizaciones por columnas parte 1.....	14
Cómo funciona.....	14
Resultados.....	15
Limitación.....	17
Posibles vías de mejora, parte 1.....	18
Tiling partiendo de la 1.1.....	19
Análisis de cómo ha de ser eficiente (partiendo de la optimización de la 1.1).....	19
ACCESOS a las caches fórmulas deducciones.....	26
Caso (Bloque origen + Tamaño array cache * 2 + Bloque destino) * 6 < L3 size.....	26
Resumen formulas memoria.....	28
Analisis codigo assembler parte interna.....	31
Estimar el BW de las caches.....	34
Resultado final de los BW.....	37
Montar y analizar la función del tiempo límite del código.....	37
La última capa de dificultad que añadiremos a la fórmula y que asumimos para que sea correcta.....	41
Optimizaciones del programa y resultados parte 2.....	45
Versión Optimizaciones por columnas parte 2.....	45
Posibles vías de mejora, parte 2.....	45

Hacer dos iteraciones en una.....	46
Cómo funciona.....	46
Resultados.....	47
Datos de rendimiento en tiempo (resumen, compilador versión reciente):.....	48
Datos globales del código directo (compilador versión reciente):.....	48
Limitaciones.....	49
Posibles vías de mejora.....	49
Conclusiones.....	50
Enlaces de recursos propios.....	50

El juego de Conway 's game of life.

Definición de la wikipedia :

The universe of the Game of Life is [an infinite, two-dimensional orthogonal grid of square cells](#), each of which is in one of two possible states, live or dead (or populated and unpopulated, respectively). Every cell interacts with its eight [neighbours](#), which are the cells that are horizontally [Square tiling - Wikipedia](#), vertically, or diagonally adjacent. At each step in time, the following transitions occur:

- 1. Any live cell with fewer than two live neighbours dies, as if by underpopulation.*
- 2. Any live cell with two or three live neighbours lives on to the next generation.*
- 3. Any live cell with more than three live neighbours dies, as if by overpopulation.*
- 4. Any dead cell with exactly three live neighbours becomes a live cell, as if by reproduction.*

The initial pattern constitutes the seed of the system. The first generation is created by applying the above rules simultaneously to every cell in the seed, live or dead; births and deaths occur simultaneously, and the discrete moment at which this happens is sometimes called a tick.^[nb 1] Each generation is a [pure function](#) of the preceding one. The rules continue to be applied repeatedly to create further generations.

Wikipedia contributors. (2025, 28 septiembre). Conway's Game of Life. Wikipedia.

https://en.wikipedia.org/wiki/Conway%27s_Game_of_Life

Dada la definición de la wikipedia:

El Juego de la Vida es un algoritmo diseñado para imitar el comportamiento de células vivas. La idea es implementar reglas simples de muerte y reproducción para observar cómo estas células computacionales muestran comportamientos que recuerdan a formas de vida.

Análisis de las características del algoritmo.

Estructuras de datos:

N = número de filas* número de columnas

Arrays (si contamos parte gráfica): Array de 8 bits (1 byte) [N],
 Array de 8 bits tmp (1 byte) [N],
 Buffer para modificar el color de los pixeles
 [N*3]

Pseudocódigo (sin contar boundary cases):

```
for j in iteraciones:
    for i in arrayCasillas:
        numVecinos = i.vecinos(arrayTMP);
        if numVecinos < 2
            i.live = 0;
        elif numVecinos > 3
            i.live = 0;
        elif numVecinos == 3
            i.live = 1;
        i.pintar(BufferColores);

swap(arrayTMP, arrayCasillas); // swap de punteros
```

Funcionamiento computacional

Estructuras de datos y disposición en memoria

El código originalmente utiliza 2 matrices de tamaño 10000x10000 con datos de tipo int. Cada una de estas matrices ocupa 10000x10000x4= 400 MB. Esto hace que el programa no quepa en ni siquiera en L3 haciendo que se tenga que escribir y leer en MM provocando un posible cuello de botella de tipo bandwidth-bound de memoria.

Núcleo básico de cómputo

El núcleo computacional principal se encuentra en las funciones juego_de_la_vida, actualizar_tablero y evaluar_celda. La función actualizar_tablero llama, para cada celda, a evaluar_celda, que contiene el bucle interno anidado más costoso. A su vez, juego_de_la_vida invoca actualizar_tablero tantas veces como indique el parámetro time.

El acceso a la memoria de evaluar_celda es el siguiente:

Supongamos que esta cuadrícula representa la matriz tablero y el número de adentro representa la cantidad de veces que se lee el dato de una posición.

Vamos a suponer que se llama a evaluar_celda para las posiciones marcadas en la celda.

Iter 1 -> Azul

Iter 2-> Verde

Iter 3 -> Rojo

1	1+1=2	1+1+1=3	1+1=2	1
1	1+1=2	1+1+1=3	1+1=2	1
1	1+1=2	1+1+1=3	1+1=2	1

Como se puede ver, el programa lee las mismas posiciones de memorias una y otra vez cuando no es necesario, puesto que este será el caso para casi todas las casillas.

Una posible manera de solucionar esto sería leer una columna y almacenar el número de vecinos vivos. Sin embargo, gracias a la información proporcionada por nuestro profesor Juan Carlos Moure, sabemos que este enfoque elimina la vectorización. Por tanto, probaremos a seguir una vía distinta.

Patrón de computo

Como se ha visto en el punto anterior, el programa accede principalmente a una posición de memoria y a sus ocho posiciones vecinas en el tablero, y guarda el resultado en otro tablero (aunque estas ocho posiciones no son todas contiguas en memoria). Esto provoca una gran localidad espacial en las columnas, pero una localidad espacial muy reducida en las filas. Además, no existe dependencia entre iteraciones. Por tanto, el patrón de cómputo corresponde a un **9-point stencil**.

Por otra parte, al inicializar el programa también aparece un patrón de tipo *scan* al generar los números aleatorios, ya que es necesario recorrer la estructura completa para inicializarla.

Vectorización

Tanto la función evaluar_celda como actualizar_tablero són funciones que presentan un patrón de computo de tipo stencil, por lo que són posiblemente vectorizables (ya hemos hecho pruebas y sabemos a ciencia cierta de que si es posible) puesto que no hay dependencias de escritura entre celdas, porque se lee de un tablero y se escribe en otro.

Complejidad algorítmica

La complejidad algorítmica es $O(N^2 * \text{iteraciones})$ debido a que haremos $X*N*N$ iteraciones de operaciones en base a que N es el número de celdas en una fila y las columnas tienen el tamaño de las filas.

Localidad temporal

La localidad temporal del Game of Life consiste en una parte como explicamos en el núcleo de cómputo básico por columnas, reutilizamos hasta 3 veces la misma columna (casi siempre) para generar cada elemento de una fila. Pero además cada fila se lee otras tres veces también, siguiendo la lógica del ejemplo de antes.

Sin embargo, más allá de esto, cada celda deja de usarse hasta la siguiente iteración completa, momento en el que ya no se encuentra en caché. Por tanto, la localidad temporal global del algoritmo es muy baja. Hay una pequeña reutilización a nivel de filas cercanas, pero no existe una utilización sostenida de los datos a lo largo del tiempo (al menos por ahora).

Intensidad aritmética

Para cada celda en cada celda se hacen las siguientes operaciones:
10 loads, 9 sumas, 1 resta, 3 comparaciones y 1 escritura. Esto da un total de 24 instrucciones.

Sí, si miramos la intensidad aritmética obtenemos:
 $24 \text{ operaciones} / ((10 + 1) \cdot 4 \text{ bytes}) \approx 0,55$.

Esto muestra que el programa está limitado por memoria (*memory-bound*), ya que gran parte del tiempo está esperando a que lleguen los datos en lugar de procesarlos.

Complejidad espacial

El programa en su totalidad necesita: 1 tablero y 1 tablero auxiliar. Puesto que los dos ocupan lo mismo y que cada tablero ocupa 10000×10000 y almacena valores de tipo int que ocupan 4 bytes cada uno, podemos ver que el programa ocupa $10000^2 \cdot 4 \cdot 2 = 800 \text{ MB}$.

Este tamaño es tan elevado que ni siquiera cabe en la memoria compartida L3, por lo que se tiene que leer y escribir en memoria principal que es más lenta. Por tanto se necesitan $O(N^2)$ elementos siendo la cantidad de filas y columnas igual, y cantidad de elementos en una fila N .

Herramienta propia de análisis del programa

Nuestra herramienta consiste en tres scripts: dos en Bash y uno en Python. Dentro del entorno del laboratorio ejecutamos mountExperiment.sh.

En este script, dentro del array experiments, definimos para cada experimento o prueba los inputs que debe tener.

Formato:

row	col	iter	threads	name
120	12000	10000	1	Entra_L1

De esta forma construimos el código que se ejecutará.

```
[aa-26@aolin-login createOneJSONexperiment]$ bash ./mountExperiment.sh
```

Dejamos enlace aquí:

[mountExperiment.sh](#)

Este script mountExperiment.sh llama repetidas veces al script createAnalisisJSON.sh, para que este genere el análisis de cada experimento de forma individual. Por motivos prácticos, cada experimento se repite solo una vez, aunque sería interesante repetirlo al menos tres veces y quedarse con el mejor resultado.

Para que el proceso funcione correctamente, es necesario incluir en la misma carpeta un archivo llamado exampleGenerate.c, en el cual se utilizarán variables globales definidas en createAnalisisJSON.sh. El fichero .c debe eliminar cualquier print y cualquier variable global que pueda colisionar con las de createAnalisisJSON. Además, en dicho archivo se añadirá **`__attribute__((noinline))`** a aquellas funciones donde no afecte al rendimiento, de forma que, pese a eliminar los print, el compilador no elimine código que considere redundante.

El script en Bash, además, añade los define necesarios y compila cada experimento del mismo modo que lo haríamos manualmente.

Dejamos enlace aquí:

[createAnalisisJSON.sh](#)

[exampleGenerate.c](#) //es un ejemplo

Andreu Elias Puigvert Gómez - 1702901
Jaime Amselem Herrero - 1704539

Tras ejecutar mountExperiment.sh, se genera un archivo JSON que posteriormente será leído por el script en Python para crear un archivo Excel a partir de estos datos. El script generateExcel.py toma el fichero result.json ubicado en su directorio y genera el Excel correspondiente.

Dejamos enlace aquí:

[generateExcel.py](#) // Importante, para uso instalar la librería **openpyxl**

Se ejecuta así:

```
andreu@Cotorra-Guapa:~/Escritorio/Uni/Tercera/AA/Proyecto$ python3 ./main.py  
andreu@Cotorra-Guapa:~/Escritorio/Uni/Tercera/AA/Proyecto$
```

Optimizaciones del programa y resultados parte 1.

Todas las tablas completas con todos los datos se encuentran aquí:

https://docs.google.com/spreadsheets/d/19obqrpNW9OFioGuNY9P__RenLfJUyo1-cVKNgMWz8ns/edit?usp=sharing

Cómo compilamos

Nosotros empezamos probando con la versión del compilador default porque resultaba ser la que nos devolvía mejor rendimiento al principio, pero a medida que implementamos las optimizaciones directas ya era mejor pivotar a la versión más nueva disponible, más nos dimos cuenta en un momento tardío pero a tiempo.

Ahora bien, nosotros compilamos con los siguientes flags con gcc:

```
gcc -Ofast -march=native -fopenmp -fopt-info-vec ./GOL.c -o GOL
```


Versión Optimizaciones directas

Cómo funciona

Puesto que el código nos venía con erratas claras para el rendimiento, con intención cabe destacar, nos dedicaremos a mencionar aquí cuáles fueron las correcciones a aplicar pero realmente carece de importancia de explicar pues son demasiado simples.

Optimizaciones básicas:

- Loop interchange
- Convertir los arrays a char
- Eliminar condicionales if
- Intercambiamos roles entre destino y origen
- Calculamos el checksum en la última iteración
- Paralelizar el código
- Cambiamos el cómo se inicializa el tablero (básicamente encontrar la manera de iterar la generación del estado inicial por mayor localidad temporal)

Resultados

Datos de rendimiento en tiempo (resumen, compilador versión reciente):

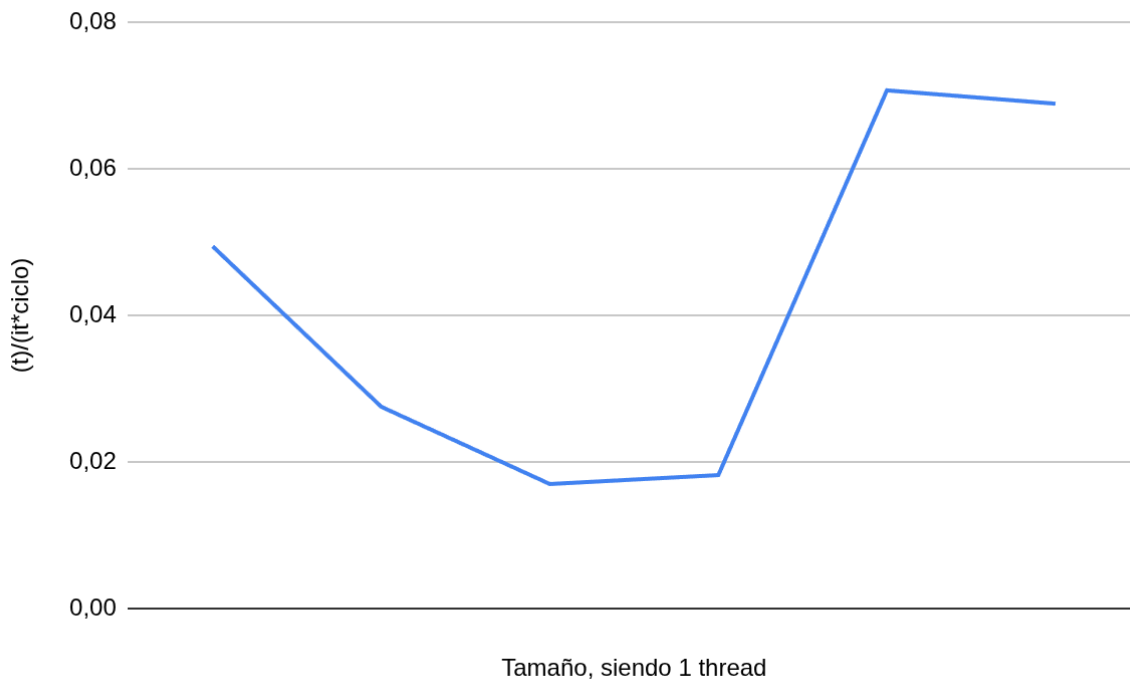
X	Y	Iteraciones	Tiempo Original	SpeedUp x 1 thread Nueva versión	SpeedUp x 6 thread Nueva versión	SpeedUp x 12 thread Nueva versión
150	150	100000	9,4	40,8x	84,1x	100,2x
750	750	10000	29,3	59,3x	189,1x	278,2x
3000	3000	1000	113,5	126,9x	742,7x	578,4x
3100	3100	1000	121,8	142,3x	696,2x	773,1x
10000	10000	100	254,0	222,3x	359,2x	348,0x
20000	20000	100	1157,1	248,6x	419,9x	388,4x
100000	1000	100	94,9	83,9x	140,3x	131,6x
200000	2000	100	596,1	130,2x	217,4x	196,8x

Datos globales del código directo (compilador versión reciente):

X	Y	Iteraciones	Ciclos (G)	Instrucciones (G)	IPC	Tiempo	Reloj	(t)/(it*ciclo)	N_threads	SpeedUp x 1 thread
150,00	150,00	100000,00	0,88	2,97	3,37	0,23	3,86	0,10	1,00	40,80
750,00	750,00	10000,00	2,02	5,60	2,77	0,49	4,10	0,09	1,00	59,32
3000,00	3000,00	1000,00	3,70	6,74	1,82	0,89	4,14	0,10	1,00	126,87
3100,00	3100,00	1000,00	3,42	7,37	2,16	0,86	4,01	0,09	1,00	142,31
10000,00	10000,00	100,00	4,60	8,12	1,77	1,14	4,03	0,11	1,00	222,29
20000,00	20000,00	100,00	19,78	32,41	1,64	4,65	4,25	0,12	1,00	248,57
100000,00	1000,00	100,00	4,79	8,10	1,69	1,13	4,25	0,11	1,00	83,95
200000,00	2000,00	100,00	19,63	32,38	1,65	4,58	4,29	0,11	1,00	130,24
										SpeedUp x 6 thread
150,00	150,00	100000,00	2,53	3,08	1,21	0,11	3,87	0,05	6,00	84,12
750,00	750,00	10000,00	3,64	5,65	1,55	0,15	3,97	0,03	6,00	189,12
3000,00	3000,00	1000,00	3,38	6,75	2,00	0,15	3,95	0,02	6,00	742,72
3100,00	3100,00	1000,00	3,85	7,39	1,92	0,17	3,91	0,02	6,00	696,23
10000,00	10000,00	100,00	13,72	8,16	0,59	0,71	3,92	0,07	6,00	359,18
20000,00	20000,00	100,00	53,40	32,44	0,61	2,76	3,96	0,07	6,00	419,89
100000,00	1000,00	100,00	13,08	8,12	0,62	0,68	3,96	0,07	6,00	140,34
200000,00	2000,00	100,00	52,63	32,42	0,62	2,74	3,95	0,07	6,00	217,43
										SpeedUp x 12 thread
150,00	150,00	100000,00	4,34	3,19	0,73	0,09	3,94	0,04	12,00	100,15
750,00	750,00	10000,00	4,92	5,65	1,15	0,11	3,98	0,02	12,00	278,18
3000,00	3000,00	1000,00	8,61	6,85	0,80	0,20	3,96	0,02	12,00	578,42
3100,00	3100,00	1000,00	6,89	7,40	1,07	0,16	3,96	0,02	12,00	773,06
10000,00	10000,00	100,00	27,95	8,19	0,29	0,73	3,96	0,07	12,00	347,98
20000,00	20000,00	100,00	112,53	32,53	0,29	2,98	3,96	0,07	12,00	388,43
100000,00	1000,00	100,00	27,55	8,17	0,30	0,72	3,96	0,07	12,00	131,61
200000,00	2000,00	100,00	117,76	32,44	0,28	3,03	3,96	0,08	12,00	196,79

Perfilado para cada caso de **6 threads**, para las librerías de omp a medida que aumentamos el tamaño del problema:

Limitación



Podemos observar que, a medida que el problema crece en tamaño, el tiempo necesario para generar cada celda muestra primero una tendencia descendente y, posteriormente, comienza a aumentar.

Este comportamiento se entiende con facilidad si lo analizamos junto con el perfilado puesto que este nos revela que al principio es costoso por los tiempos de coordinación.

Perfilado para **6 threads**, de las librerías de omp a medida que aumentamos el tamaño del problema:

49.35%
7.55%
3.01%
3.48%
1.31%
0.82%

Entonces, ¿Qué causa que al aumentar el tamaño el rendimiento sea menor?

Para poder llegar a entender porque esto podría ocurrir primero dibujaremos el algoritmo. Aquí observamos que un stencil-9 para generar dos nodos necesita todos los datos en la imagen:

a	b	c	j
d	e	f	k
g	h	i	i

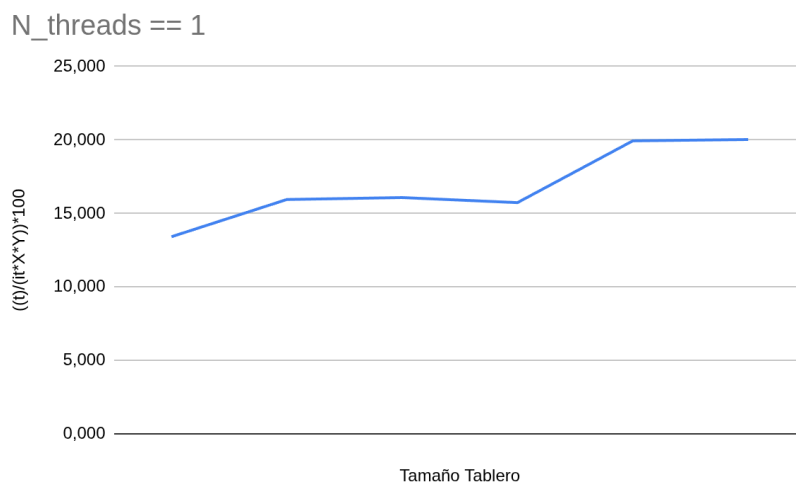
Ahora bien, una vez terminada de generar esta fila, el thread pasará a la siguiente:

d	e	f	k
g	h	i	l
m	n	o	p

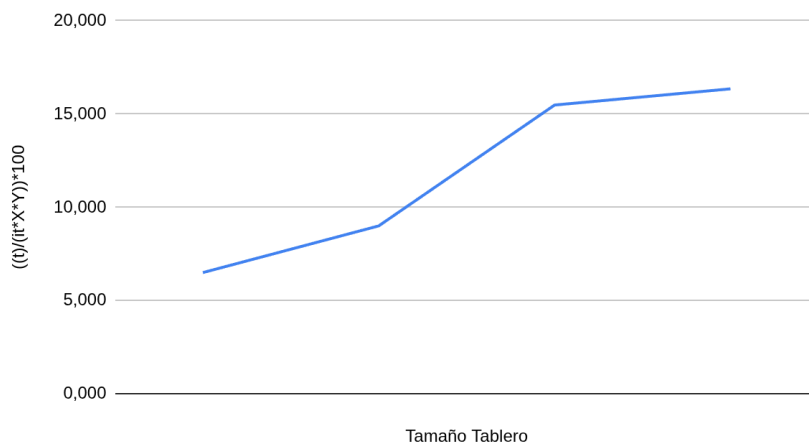
¿Qué está ocurriendo? Como podemos observar, los valores necesarios para generar *h* pueden reutilizarse en gran medida si los elementos leídos de la fila anterior permanecen en caché. Sin embargo, cuando la fila es demasiado larga, este efecto de reutilización se pierde. Para comprobarlo, conviene realizar un “pequeño” experimento: forzar que el tamaño de la fila sea tan grande que no quepa primero en la caché L1, después en L2 y, finalmente, tampoco en L3.

Aun así, este no es el único problema: al dividir el trabajo por filas, no se explota plenamente la localidad de los datos, lo que degrada todavía más el rendimiento.

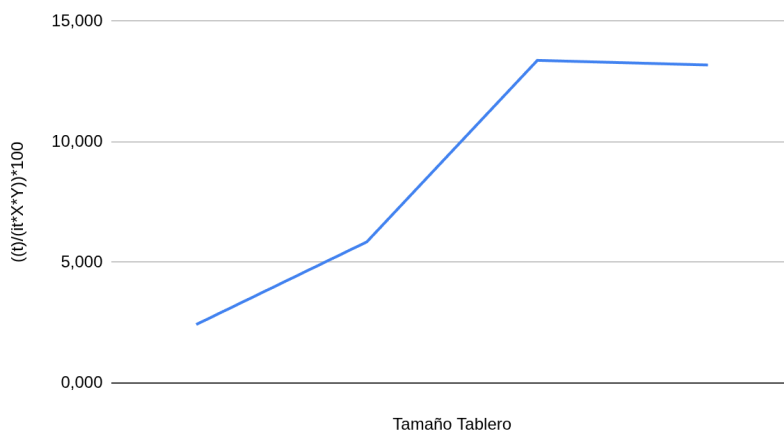
Por ello, aquí tenemos unas pruebas donde aumentamos mucho las columnas:



N_threads == 2



N_threads == 6



Cual es el problema pues, cuando el problema entra L1, L2 y L3 hasta MM no varía mucho. Pero a medida que aumentamos más allá el problema nos encontramos que empeora. Es debido esto como habíamos comentado antes por no poder hacer uso de la reutilización de filas. Además de que al repartir el trabajo por filas y no por columnas aprovechamos menos aún este reuso. Pero como evidenciamos esto mismo: en primer lugar al aumentar el tamaño del problema más allá de que no entre en L3 sigue empeorando. Al aumentar en exceso las columnas pese a ser el mismo número de elementos totales empeora el rendimiento. ¿Cuánto?

El speed up para cuando el número de columnas es menor respecto a los casos con más columnas con la misma cantidad de elementos es:

Para un solo thread.

Speed Up
0,8906370204
0,9578156748
0,9158944732
0,939632462

Ah, pero para 6

9,345247813
10,79461663
12,36551324
12,69324383

y para 12

6,954595653
12,69318857
12,74892514
13,15767047

Resumen, ciertamente al aumentar el número de columnas reducimos el rendimiento del programa considerablemente si está en paralelo, debido a que a nuestro programa le desaparece su ecología. Es decir que por cada celda a generar leemos 3 celdas de memoria y transferimos dos veces los datos de una.

Posibles vías de mejora

La más bruta posible sería hacer uso de un bitmap, el problema es que vectorizar se convierte en un dolor de cabeza y realmente no resolvería el problema, solo lo retrasaría pero llegaría a pasar lo mismo. Por tanto tenemos que buscar volvernos más ecológicos, desgraciadamente, como en la vida real es muy caro (difícil de realizar).

Por ello y para buscar este reuso podríamos en vez de generar una iteración generemos dos de golpe, así nos ahorramos accesos a M.M. Esto puede parecer una mejora, pero tiene el problema de que aumenta el número de filas necesarias a guardar en caché.

Por tanto, una solución de verdad tiene que si o si mantener la ecología de los datos, como por ejemplo dividir el problema de una manera diferente. Tan solo si hacemos que los threads dividan el trabajo en columnas a realizar podemos resolverlo. Puesto que así podemos forzar que este rehusó siempre sea en L1.

La forma de obtener el tamaño de columna es literalmente dividir el tamaño de la cache entre 4:

Entran 4 columnas		
Caches	Tamaño (bytes)	Resultado Matriz (sólo la columna)
Entra en L1	49152	12288
Entra en L2	1310720	327680
Entra en L3	18874368	4718592

Versión Optimizaciones por columnas parte 1

Cómo funciona

La versión antigua se veía así de como repartimos los trabajos, les damos a los threads filas a hacer y ya está:

```
void __attribute__((noinline)) actualizar_tablero_old ( unsigned char*
tablero_original, unsigned char* tablero_tmp )
{
    #pragma omp parallel for
    for (int i = 1; i < Y+1; i++) {
        for (int j = 1; j < X+1; j++)
            tablero_tmp[i*(2+X)+j] = evaluar_celda_XD(tablero_original, i, j);
    }
}
```

Ahora, si modificamos un poco la función para que el reparto consista en dar N columnas a hacer hasta finalizar todas las filas de estas columnas, podemos forzar a que entren las 4 columnas en caché para que podamos reutilizar la mayoría de datos, no se complica mucho, pero sí que se modifica bastante. El tema es que la mejora solo se nota si el problema tiene la suficiente cantidad de columnas. Por ello, hemos modificado un poco el código para que permita estos tamaños reservando ya memoria dinámica con malloc y hemos forzado a que X y Y sean long int:

```
#define X 100000L
#define Y 100000L
```

Y ahora si, ponemos la versión de actualizar_tablero nueva:

```
void __attribute__((noinline)) actualizar_tablero ( unsigned char*
tablero_original, unsigned char* tablero_tmp )
{
    int tid = omp_get_thread_num();
    int bloque = 10000; // tamaño del trozo horizontal
    int repartBlock = (X+1)/NUM_THREADS;
    if (repartBlock < bloque)
    {
        bloque = repartBlock;
    }
    for (int start = tid * bloque + 1; start < X+1; start += NUM_THREADS
* bloque) {
        int end = start + bloque;
        if (end > X+1) end = X+1;
        for (int i = 1; i < Y+1; i++) {
            for (int j = start; j < end; j++)
```

```
    tablero_tmp[i*(2+X)+j] = evaluar_celda_XD(tablero_original, i,  
j);  
    }  
    }  
}
```

Resultados

Por ello, ahora las pruebas serán de tamaños todavía más grandes, puesto que la idea es llegar al 1000000x1000000. Ahora bien, para ver cuanto mejora en programas grandes este ligero cambio hemos hecho esta prueba:

Actualizar tablero antiguo (gcc 8.X.X):

```
[aa-26@aolin-login proyecto]$ make time  
gcc -Ofast -march=native -fopenmp -msse4.2 -mpopcnt ./GOL.c -o GOL  
./GOL.c: En la función 'actualizar_tablero':  
./GOL.c:65:13: aviso: declaración implícita de la función 'omp_get_thread_num' [-Wimplicit-function-declaration]  
    int tid = omp_get_thread_num();  
              ^~~~~~  
perf stat ./GOL  
Matrix is 100000 x 100000.  
Number of iterations: 100  
Checksum = 739936371  
  
Performance counter stats for './GOL':  
  
    1.367.802,87 msec task-clock:u      #    5,608 CPUs utilized  
                     0 context-switches:u      #    0,000 /sec  
                     0 cpu-migrations:u      #    0,000 /sec  
         6.365.882 page-faults:u      #    4,654 K/sec  
    5.416.889.710.750 cpu_core/cycles:u/      #    3,960 G/sec  
   10.882.372.217.127 cpu_core/instructions:u/ #    7,956 G/sec  
    196.241.393.120  cpu_core/branches:u/      #   143,472 M/sec  
     64.038.926    cpu_core/branch-misses:u/ #    46,819 K/sec  
  
    243,892907120 seconds time elapsed
```

Actualizar tablero nuevo (gcc 8.X.X):

```
[aa-26@aolin-login proyecto]$ make time  
./GOL.c: En la función 'actualizar_tablero':  
./GOL.c:65:13: aviso: declaración implícita de la función 'omp_get_thread_num' [-Wimplicit-function-declaration]  
    int tid = omp_get_thread_num();  
              ^~~~~~  
perf stat ./GOL  
Matrix is 100000 x 100000.  
Number of iterations: 100  
Checksum = 739936371  
  
Performance counter stats for './GOL':  
  
    419.297,72 msec task-clock:u      #    4,533 CPUs utilized  
                     0 context-switches:u      #    0,000 /sec  
                     0 cpu-migrations:u      #    0,000 /sec  
         4.792.102 page-faults:u      #   11,429 K/sec  
    1.658.265.102.271 cpu_core/cycles:u/      #    3,955 G/sec  
    1.947.444.807.621 cpu_core/instructions:u/ #    4,645 G/sec  
     45.240.048.160  cpu_core/branches:u/      #   107,895 M/sec  
    282.631.979    cpu_core/branch-misses:u/ #   674,060 K/sec  
  
    92,506562087 seconds time elapsed  
  
    412,338771000 seconds user
```

Con esto podemos concluir que sí que ha sido una buena optimización este cambio puesto que es 2,64x (gcc 8.X.X) veces más rápido.

Versión gcc más moderna, resumen de resultados:

X	Y	Iteraciones	Tiempo Código directo	SpeedUp
150	150	100000	1,24	1,98
750	750	10000	2,45	6,24
3000	3000	1000	1,54	4,55
3100	3100	1000	1,67	4,15
10000	10000	100	2,10	2,05
20000	20000	100	7,01	1,92
20000	100000	100	36,93	2,09
100000	100000	100	197,02	2,34

Versión gcc más moderna, todos los experimentos:

X	Y	Iteraciones	Ciclos (G)	Instrucciones (G)	IPC	Tiempo	Reloj	(t)/(it*ciclo)	N_threads	SpeedUp x 1 thread
150,00	150,00	100000,00	1,13	4,39	3,88	0,29	3,90	0,13	1,00	0,95
750,00	750,00	10000,00	2,27	6,39	2,82	0,57	3,99	0,10	1,00	0,95
3000,00	3000,00	1000,00	3,35	6,97	2,08	0,83	4,05	0,09	1,00	0,91
3100,00	3100,00	1000,00	3,67	7,66	2,09	0,92	4,00	0,10	1,00	0,88
10000,00	10000,00	100,00	5,15	7,91	1,54	1,28	4,04	0,13	1,00	0,90
20000,00	20000,00	100,00	21,79	31,65	1,45	5,14	4,24	0,13	1,00	0,96
20000,00	100000,00	100,00	111,50	158,25	1,42	26,10	4,27	0,13	1,00	0,93
100000,00	100000,00	100,00	571,70	791,44	1,38	134,36	4,26	0,13	1,00	0,89
										SpeedUp x 6 thread
150,00	150,00	100000,00	14,87	22,86	1,54	0,63	3,97	0,28	6,00	1,98
750,00	750,00	10000,00	9,27	18,79	2,03	0,39	3,96	0,07	6,00	6,24
3000,00	3000,00	1000,00	7,70	8,97	1,17	0,34	3,92	0,04	6,00	4,55
3100,00	3100,00	1000,00	9,19	9,83	1,07	0,40	3,92	0,04	6,00	4,15
10000,00	10000,00	100,00	21,75	8,55	0,39	1,02	3,94	0,10	6,00	2,05
20000,00	20000,00	100,00	76,60	33,44	0,44	3,65	3,96	0,09	6,00	1,92
20000,00	100000,00	100,00	370,73	167,12	0,45	17,66	3,96	0,09	6,00	2,09
100000,00	100000,00	100,00	1519,56	792,18	0,52	84,36	3,95	0,08	6,00	2,34
										SpeedUp x 12 thread
150,00	150,00	100000,00	40,18	50,54	1,26	0,84	3,98	0,38	12,00	3,11

750,00	750,00	10000,00	26,70	36,63	1,37	0,56	3,98	0,10	12,00	8,64
3000,00	3000,00	1000,00	19,59	16,03	0,82	0,43	3,97	0,05	12,00	18,36
3100,00	3100,00	1000,00	16,81	10,88	0,65	0,37	3,96	0,04	12,00	6,41
10000,00	10000,00	100,00	46,81	9,03	0,19	1,14	3,95	0,11	12,00	2,08
20000,00	20000,00	100,00	171,43	34,29	0,20	4,12	3,96	0,10	12,00	2,23
20000,00	100000,00	100,00	822,73	171,14	0,21	19,68	3,96	0,10	12,00	4,51
100000,00	100000,00	100,00	3083,68	830,58	0,27	88,25	3,95	0,09	12,00	5,49

Limitación

Primero obtenemos cuantos fallos en caché tenemos (una aproximación con perf):

```

~~~~~
perf stat -e cycles,instructions,branches,branch-misses,cache-references,cache-misses,LLC-loads,LLC-l
Matrix is 100000 x 100000.
Number of iterations: 100
Checksum = 739936371

Performance counter stats for './GOL':

1.679.936.134.991      cpu_core/cycles:u/                (30,00%)
1.915.207.747.786      cpu_core/instructions:u/          (40,00%)
 41.827.362.407        cpu_core/branches:u/              (50,00%)
  99.271.136           cpu_core/branch-misses:u/         (60,00%)
25.000.227.176         cpu_core/cache-references:u/      (70,00%)
21.906.355.033         cpu_core/cache-misses:u/         (80,00%)
 442.278.746          cpu_core/LLC-loads:u/             (80,00%)
 344.797.618          cpu_core/LLC-load-misses:u/       (80,00%)
 629.528.126          cpu_core/LLC-stores:u/            (20,00%)
 499.718.466          cpu_core/LLC-store-misses:u/     (20,00%)

 92,608891264 seconds time elapsed

414,844945000 seconds user
  5,401111000 seconds sys

```

El total **aproximado** de bytes transferidos sabiendo que el tamaño de los bloques es 64 bytes en esta arquitectura es:

1305,72 GiBytes

El tamaño de total en bytes de los dos arrays es:

9,31 GiBytes

Es decir, **aproximadamente** leemos 140,2x veces los arrays.

Siendo el array “x*y” y “n” el número de iteraciones.

Total de bytes transferidos de memoria principal al procesador = tamaño de los dos arrays * n iteraciones + (escritura del array) tamaño de uno de los dos arrays * n iteraciones = $x*y*2*n + x*y*n = 3*x*y*n$

Sabiendo que el tamaño de bloque es 64 bytes

Misses L3 = $3*x*y*n / 64$

Andreu Elias Puigvert Gómez - 1702901
Jaime Amselem Herrero - 1704539

Total para calcular si es memory bound en M.M:

$$\text{tiempo de ejecución} = \frac{3 \times x \times y \times n}{\text{Bandwidth M.M}}$$

En nuestro caso (sabiendo que el cálculo es aproximado y salvando las distancias)

$$92,6 = \frac{3 \times 100000 \times 100000 \times 100}{\text{BW_M.M}} \rightarrow \text{BW_M.M} = \frac{3 \times 100000 \times 100000 \times 100}{92,6} \\ = 30,17 \text{ GB/s debería tener en global la memoria si es lo que nos limita.}$$

Ahora, hacemos unas pruebas para saber el bandwidth:

Usamos un comando en bash que testea la transferencia de un 1GB:

```
seq 0 5 | parallel -j6 taskset -c {} perf bench mem memcopy --size 1GB
```

Ahora, el out nos da con memcopy() que lee y copia, por tanto sumamos todas las instrucciones por segundo que ejecuta de default que son (de cada core):

$$\text{total} = 2,62 + 2,553 + 2,48 \times 2 + 2,47 + 2,47 = 15 \text{ GB memcopy / s}$$

$$\text{memcopy} = 1 \text{ Lectura} + 1 \text{ escritura} = 2 \text{ operaciones en M.M}$$

$$\text{BW M.M.} = 15,073 \times 2 = 30 \text{ GB / s}$$

Que es lo que estimamos que sería el B.W. si nos limitará la memoria.

Posibles vías de mejora, parte 1

Debido a que ahora estamos llegando a estos puntos, toca comenzar a ser un poco más creativos. Las opciones creativas que tenemos son las siguientes:

Si realmente limita memoria intentar un bitmap vectorizable:

Pros:

Reducimos entre entre 8 la cantidad de memoria total del programa

Contras:

Difícil de implementar

Podría no ser rentable (hay que hacer más operaciones para tratar con bitmaps)

Solo sabemos a nivel teórico que se puede vectorizar, tal vez el compilador no lo entienda y haya que hacer assembler

Repartir en vez de bloques de columnas a hacer, repartir por bloques que entren en memorias más rápidas y generar el resultado hasta n iteraciones.

Pros:

Nos garantizamos un mayor reuso de datos

Implementación más sencilla

Contras:

Creamos redundancia (se podría solucionar pero no se hará al inicio)

Hacer dos iteraciones en una, si lo hacemos bien leemos la mitad de datos en memoria DDR4:

Pros:

Nos garantizamos una menor transferencia de datos con la memoria principal.

Contras:

Creamos redundancia

Resumen:

	Possible speed up para 100000x100000 it = 100 si siempre memoria DDR5 el límite	Dificultad de vectorizar(0-10)	Dificultad de implementar
Compresión bitmap	8x (si sigue limitando el cómputo)	10	10
Tiling	¿?	0	9
Dos iteraciones en una	2x	5	7,5

Antes de empezar a saber cuál vía tomar, lo lógico sería saber cómo de beneficioso sería hacer tiling, puesto que si la diferencia es sustancial sería mucha mejor opción.

Tiling partiendo de la 1.1

Análisis de cómo ha de ser eficiente (partiendo de la optimización de la 1.1)

Debido a que para que partimos de la base de que queremos a futuro trabajar para que a futuro la RAM no sea un problema para cuando hagamos nuestro objetivo 1000000x1000000, no podemos partir por bloques en cada iteración y ya, puesto que no entran en L3 bajo ningún concepto. Por ello necesitamos coger un trozo del problema y trabajar lo más a futuro con el máximo beneficio en rendimiento de manera que reduzcamos el uso de la RAM.

¿ Cómo empezamos ? Fácil, hablemos un poco de como funciona el game of life, dada cada iteración se actualizan todos los nodos. Entonces se leen de esta matriz resultante y se vuelve a generar. Este proceso se puede visualizar de la siguiente forma:

1	1	1		
1		1	----->	2
1	1	1		

2	2	2			
2	2	2	----->	----->	3
2	2	2			

Es decir, para generar la primera iteración (los nodos 2), miran la posición en la iteración actual, sus vecinos y generan el resultado en otra matriz. Luego este paso se repite con estos nodos de tipo dos a los de la segunda iteración, tipo 3. En todo este proceso, hemos necesitado ir a M.M para obtener los 1, escribir (que también implica leer) los 2, de nuevo los 2 y leer de nuevo de M.M para escribir (que también implica leer) el 3.

Ahora bien, ¿ Partiendo de la matriz antes de aplicar la iteración uno cuantos nodos hemos necesitado para aplicar la 2? La manera es muy directa, tomamos los vecinos de los nodos vecinos 1 del nodo 3 a generar obteniendo una tabla 5x5 :

1		1		1		1		
1		1		1		1		
1		1		1		1	----->	3
1		1		1		1		
1		1		1		1		

Como ya se ha podido observar para generar un único valor hemos de tomar muchos datos. Por ello, si queremos ir desde una matriz origen a una matriz objetivo, la matriz origen ha de ser más grande que la matriz objetivo. Aún así, como muchos de estos datos los habríamos de haber leído igual si nos saltamos el salto entre medio iremos mucho más rápido pues pediremos la mitad de datos a M.M.

Esto nos da pie a definir los siguiente, que siendo n iteraciones y mxp la matriz objetivo (quitando boundary cases):

- Si generamos hasta n iteraciones hacia adelante necesitaremos una matriz origen de $(m+2n) \times (p+2n)$.
- El total de operaciones de una iteración se define en este apartado como el total de actualizaciones de celda. Por tanto esta norma dado un array mxp es $(m-1) \times (p-1)$.

- El total de operaciones se define como:

$$\sum_{x=1}^{n-1} [(m + 2x)(p + 2x)] + 1$$

- Debido a que de momento no hacemos dos iteraciones de golpe ni nada semejante, cada iteración ha de ser calculada, por tanto los cálculos de los nodos bordes podríamos terminar repitiendo los mucho, es decir haremos por cada tarea de

$$\text{blocking} \sum_{x=1}^{n-1} ((m+2x)(p+2x)-(m)(p)) \text{ operaciones de más o en texto:}$$

array origen tamaño a generar en la siguiente iteración - tamaño destino y sumar para cada iteración.

- El porcentaje de operaciones útiles se define por :

$$\left(1 - \frac{\sum_{x=1}^{n-1} [(m+2x)(p+2x)-(m)(p)]}{\sum_{x=1}^{n-1} [(m+2x)(p+2x)]+1}\right) * 100$$

Entonces, ya habiendo definido esto, nos encontramos en un compromiso. Por una parte intentar reproducir el problema pero solo en L1 parece tentador.

Ahora bien, los tamaños son “pequeños” y todavía no sabemos si compensa todo el exceso de operaciones para generar matrices tan pequeñas. Puesto que las grandes tienen a nivel porcentual menos exceso, pero ¿cuánto?

Vamos a calcularlo :

Para ello, primero calculamos si las columnas y las filas son iguales el límite cuando entran las matrices en caché:

Esto lo calculamos de la siguiente forma:

TM = tamaño memoria
C = tamaño columna

Si queremos que esté al límite y sabiendo que usamos dos matrices :

$$TM = 2 * C^2$$

Aislamos y nos quedamos con la fórmula:

$$\sqrt{(TM/2)} = C$$

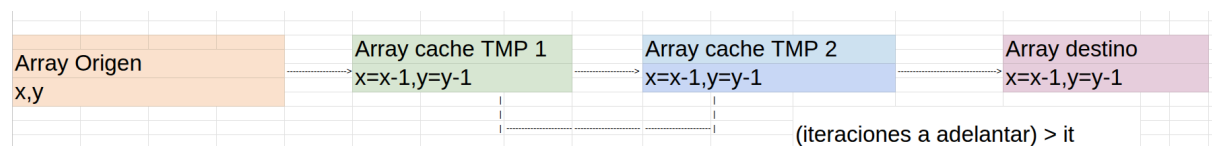
Entra matriz		
Caches	Tamaño (bytes)	Resultado Matriz (sólo la columna)
Entra en L1	49152	156,7673435
Entra en L2	1310720	809,543081
Entra en L3	18874368	3072

Ahora, ya sabemos calcular el exceso, como calcular el total de operaciones, etc. Por tanto nos queda estimar cuánto sería el speed up en cada caso y obviamente tomar el máximo haciendo pruebas para ver si vamos a 5, 10, 20, 30, 40, 50 iteraciones hasta volver a regenerar las tareas.

Problema, para hacer esto, tenemos que tener dos matrices temporales donde leer y escribir para evitar race condition etc, por tanto (sino entrarán en L3) tendríamos que leer de memoria. Por tanto, hemos de buscar un equilibrio muy interesante.

Ahora, vamos a ver cuando la cantidad de accesos a M.M se reduce y si sale rentable:

Primero de todo limitaremos el tamaño de los array temporales para que al menos todos entren en L3 para 6 threads. Que deje espacio para mantener estos vectores temporales. Para entender lo que decimos dibujemos el problema:



Como se puede observar, tomamos un bloque del array original. Este bloque a cada paso se va reduciendo hasta generar la última iteración y guardarlo en el array final que luego usaremos como array original.

Entonces, aquí nos ocurren diversos casos:

Mantenemos los array caché en caché:

Bloque origen + Tamaño array cache * 2 + Bloque destino < L1 size

Bloque origen + Tamaño array cache * 2 + Bloque destino < L2 size

Bloque origen + Tamaño array cache * 2 + Bloque destino < L3 size

No Mantenemos los array caché en caché así a cada vez que tenemos que tomar un bloque origen para generar destino hay que llamar a M.M :

Tamaño array cache * 2 < L1 size

Tamaño array cache * 2 < L2 size

Tamaño array cache * 2 < L3 size

Total, cuantos accesos a M.M. es lo que nos interesa, así que vamos a ello ,siendo n iteraciones a calcular con el bloque, $m \times p$ la matriz objetivo, total de iteraciones globales i , y para las matrices del problema será $x \times y$ de tamaño:

Caso (**Bloque origen + Tamaño array cache * 2 + Bloque destino**)*6 < L3 size:

Tamaño array cache solo se carga en M.M una vez y por tanto suponemos que en promedio es 0 los bytes transferidos:

Explicación fórmulas:

$$\text{total de bloques} = \left(\frac{xy}{mp} \right)$$

$$\text{Total de veces que leer en M.M. el array grande} = \frac{i}{n}$$

total de bytes que se transferirán de M.M los bloques de las matrices del problema

$$\left(\frac{xy}{mp} \right) * \frac{i}{n} ((m + 2 * n)(p + 2 * n) + (m)(p) * 2 (R y W))$$

Limitaciones:

$$((m + 2 * n)(p + 2 * n) + (m)(p) + (m + 2 * (n - 1))(p + 2 * (n - 1)) + (m + 2 * (n - 2))(p + 2 * (n - 2))) * 6 < 18874368$$

El *6 es porque suponemos el uso de 6 threads

$$n < i$$

$$mp < xy$$

$$n < m$$

$$n < p$$

Caso (**Tamaño array cache * 2**) * 6 < L3 size :

$$\left(\frac{xy}{mp} \right) * \frac{i}{n} ((m + 2n)(p + 2n) + 2 (R y W) * (m)(p) + (m + 2(n - 1))(p + 2(n - 1)) * 2(R y W) + (m + n - 2)(p + n - 2) * 2(R y W))$$

Limitaciones:

$$((m + 2(n - 1))(p + 2(n - 1)) + ((m + 2(n - 2))(p + 2(n - 2))) * 6 < 18874368$$

(siendo 6 el número de threads)

Si recordamos, la cantidad transferida anteriormente es $3 \times x \times y \times n = 3 \times y \times x \times i$

Vale, entonces tenemos los límites, los cálculos para saber cuánta mejora hay, etc. Solo falta encontrar el punto óptimo para n,m y p. Primero de todo vamos a intentar reducir las expresiones al mínimo posible y luego obtener el mejor caso:

Caso (**Bloque origen + Tamaño array cache * 2 + Bloque destino**) * 6 < L3 size :

$$\left(\frac{xy}{mp}\right) * \frac{i}{n} ((m + 2n)(p + 2n) + 2 * (m)(p)) =$$

$$x * y * i * \left(\frac{3}{n} + \frac{2}{m} + \frac{2}{p} + \frac{4n}{mp}\right)$$

Limitaciones:

$$((m + 2 * n)(p + 2 * n) + (m)(p) + (m + 2 * (n - 1))(p + 2 * (n - 1)) + (m + 2 * (n - 2))(p + 2 * (n - 2))) * 6 < 18874368$$

$$((m + 2 * n)(p + 2 * n) + (m)(p) + (m + 2 * (n - 1))(p + 2 * (n - 1)) + (m + 2 * (n - 2))(p + 2 * (n - 2))) < 3145728$$

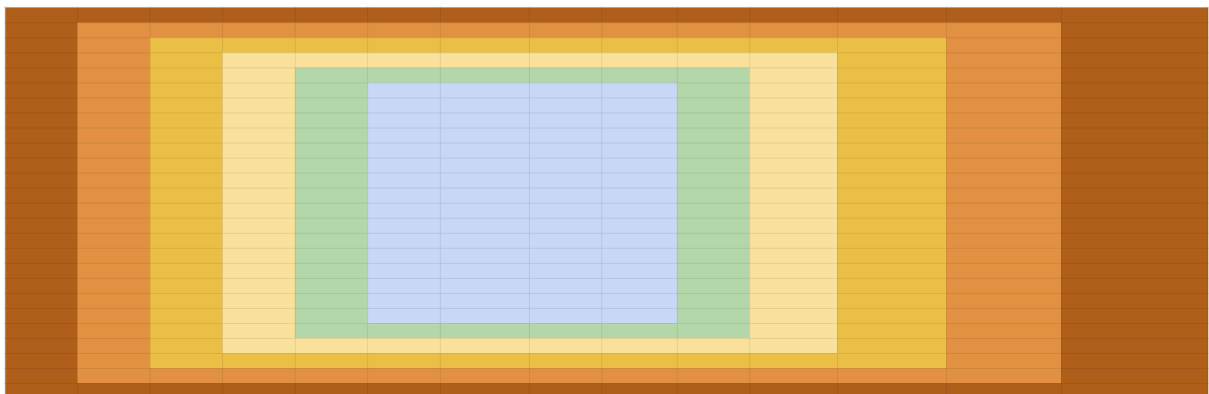
$$n < i$$

$$mp < xy$$

$$n < m$$

$$n < p$$

Antes de seguir nos gustaría compartir un poco como es el problema en este caso para visualizar como las llamadas a memoria principal son tan pocas y porque funciona:



Lo que se ve en la imagen es sencillo, cada color es una iteración, son 5 iteraciones y el array final es un 10x10 que sería lo que guardaremos en array global (imaginemos que esos son los tamaños).

¿Por qué es mejor? Bueno si, al principio tomamos un 20x20 pero lo demás lo tenemos en caché y solo usamos memoria cuando escribimos (y leemos por ende) el 10x10.

Mientras tanto, de la forma original para generar este 10x10 hasta las 5 iteración habríamos tenido que hacer 10x10 (de lectura puesto que prácticamente solo leemos una vez de M.M

Andreu Elias Puigvert Gómez - 1702901
Jaime Amselem Herrero - 1704539

para generar la siguiente iteración en la 1.1) y escribir un 10x10 (por ende también leer) y estos 3 10x10 5 veces. Resultado

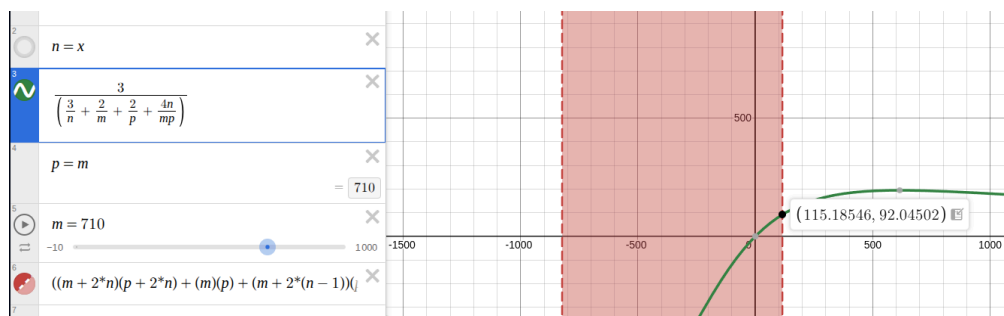
M.M en 1.1 = $10 \cdot 10 \cdot 3 \cdot 5 = 1500$ bytes

M.M en 1.2 = $20 \cdot 20 + 10 \cdot 10 \cdot 2 = 600$ bytes

La 1.2 usa 2,5x menos accesos a M.P.

Resumen, ya en casos pequeños es una diferencia notoria, y como ya se puede observar el beneficio del rendimiento viene más dado por la n que cualquier otra variable como se puede apreciar en la fórmula donde el valor de ésta al aumentar reduce los accesos a M.P. :

$$\frac{3 \cdot x \cdot y \cdot i}{x \cdot y \cdot i \cdot \left(\frac{3}{n} + \frac{2}{m} + \frac{2}{p} + \frac{4n}{mp} \right)} = \frac{3}{\left(\frac{3}{n} + \frac{2}{m} + \frac{2}{p} + \frac{4n}{mp} \right)}$$



Como se puede observar, la cantidad de accesos a memoria se reduce. Más, hay un pago, mucho más cálculo. Pero aún nos estamos adelantando.

ACCESOS a las caches

fórmulas deducciones

Vale, como hemos visto mirando solo a M.P. ahora toca L3, L2, L1:

Caso (Bloque origen + Tamaño array cache * 2 + Bloque destino) * 6 < L3 size

Para L3:

Como en M.P hay diversos casos:

- (Bloque origen + Tamaño array cache * 2 + Bloque destino) < L1 size
- (Bloque origen + Tamaño array cache * 2 + Bloque destino) < L2 size
- (Bloque origen + Tamaño array cache * 2 + Bloque destino) > L2 size

De estos tres, dos nos dan las mismas fórmulas :

- (**Bloque origen + Tamaño array cache * 2 + Bloque destino**) < L1 size
- (**Bloque origen + Tamaño array cache * 2 + Bloque destino**) < L2 size

Cuando esto se cumple nos pasa como en M.P. en L3, solo accedemos cuando hay que escribir o leer de los tableros del juego, por tanto la fórmula es la misma:

$$x * y * i * \left(\frac{3}{n} + \frac{2}{m} + \frac{2}{p} + \frac{4n}{mp} \right) \text{ (Bytes)}$$

Ahora bien, en el **caso de (Bloque origen + Tamaño array cache * 2 + Bloque destino) > L2 y (Tamaño array cache * 2) > L2 size** tenemos que formular los accesos para L3:

Este trozo es así porque estamos acumulando las veces que leemos de L3 los bloques, obviamente, como vamos reduciendo la matriz que solicitamos al ir iterando este comportamiento se refleja así en esta fórmula (luego más adelante la simplificamos). Ahora bien el 3* indica que al escribir en el bloque se tendrá (basándonos en write-allocate) que transferir dos veces (la lectura y la escritura), y el +1 es la lectura para generar la iteración posterior.

$n-1$

$$\sum_{x=1} [3(m + 2x)(p + 2x)]$$

La fórmula resultante es esta:

$$\left(\frac{xy}{mp}\right) * \frac{i}{n} ((m + 2n)(p + 2n) + 2(R y W) * (m)(p) +$$

$$\sum_{x=1}^{n-1} [3(m + 2x)(p + 2x)]) \text{ (Bytes)}$$

Ahora bien, en el caso de (**Bloque origen + Tamaño array cache * 2 + Bloque destino**) > **L2** y (**Tamaño array cache * 2**) < **L2 size** tenemos que formular los accesos para L3:

$$\left(\frac{xy}{mp}\right) * \frac{i}{n} ((m + 2n)(p + 2n) + 2 * (m)(p) +$$

$$(m + 2(n - 1))(p + 2(n - 1)) * 2 +$$

$$(m + 2(n - 2))(p + 2(n - 2)) * 2)$$

Ahora para L2 será exactamente lo mismo, por tanto ya estaría por aquí. L1 será un poco diferente excepto el caso **caso de (Bloque origen + Tamaño array cache * 2 + Bloque destino) > L1 y (Tamaño array cache * 2) > L1 size** que se comporta igual que los demás niveles.

En L1 cuando (Bloque origen + Tamaño array cache * 2 + Bloque destino) < L1 size:

$$\left(\frac{xy}{mp}\right) * \frac{i}{n} ((m + 2n)(p + 2n) + 2(R y W) * (m)(p) +$$

$$\sum_{x=1}^{n-1} [2(m + 2x)(p + 2x)]) \text{ (Bytes)}$$

El *3 de antes se convierte en *2 porque no tenemos que hacer write-allocate en este caso concreto más que al final.

En L1 cuando (Bloque origen + Tamaño array cache * 2 + Bloque destino) > L1 size y (Tamaño array cache * 2) < L1 size:

La explicación de la fórmula es la siguiente:

- Al estar en caché los bloques cuando se leen la primera vez al empezar a utilizarlos tenemos que poner un +1 extra para estos dos casos de ahí:

$$(m + 2(n - 1))(p + 2(n - 1)) + (m + 2(n - 2))(p + 2(n - 2))$$

- Coger la parte origen y escribir la parte destino de las matrices grandes:

$$(m + 2n)(p + 2n) + 2(R y W) * (m)(p)$$

- Como las matrices están en L1 solo leemos y escribimos una vez con ellas, ya que el write-allocate en L1 si ya está en L1 no nos afecta, por tanto ahora en vez del *3 tenemos ese *2.

$n-1$

$$\sum_{x=1}^{n-1} [2(m + 2x)(p + 2x)]$$

Resultado de formula:

$$\left(\frac{xy}{mp}\right) * \frac{i}{n} ((m + 2n)(p + 2n) + 2(R y W) * (m)(p) + (m + 2(n - 1))(p + 2(n - 1)) + (m + 2(n - 2))(p + 2(n - 2)) +$$

$$\sum_{x=1}^{n-1} [2(m + 2x)(p + 2x)] \text{ (Bytes)}$$

Definidos ahora los niveles, ya tenemos todas las fórmulas, vamos a resumirlas.

Resumen formulas memoria

Esta parte de la inecuación:

(Bloque origen + Tamaño array cache * 2 + Bloque destino) == core problem == CP

(Tamaño array cache * 2) == core problem == CP2

Num_threads=6=NT

Uso de recursos de	Fórmula de bytes transferidos
Memoria Principal (DDR4) CP2*NT < L3_size < CP*NT (L3 -> solo le entran los 2 bloques personales)	$\left(\frac{xy}{mp}\right) * \frac{i}{n} ((m + 2n)(p + 2n) + 2(R y W) * (m)(p) + (m + 2(n - 1))(p + 2(n - 1)) * 2(R y W) + (m + n - 2)(p + n - 2) * 2(R y W)) \text{ (Bytes)}$
Memoria Principal (DDR4) CP*NT < L3 size	$x * y * i * \left(\frac{3}{n} + \frac{2}{m} + \frac{2}{p} + \frac{4n}{mp}\right) \text{ (Bytes)}$
L3, cuando L2_size < CP2 y CP2*NT < L3_size (Los arrays personales solo entran en L3)	$\left(\frac{xy}{mp}\right) * \frac{i}{n} ((m + 2n)(p + 2n) + 2(R y W) * (m)(p) + \sum_{x=1}^{n-1} [3(m + 2x)(p + 2x)]) \text{ (Bytes)}$
L3, cuando CP2 < L2_size < CP	$\left(\frac{xy}{mp}\right) * \frac{i}{n} ((m + 2n)(p + 2n) + 2 * (m)(p) + (m + 2(n - 1))(p + 2(n - 1)) * 2 + (m + 2(n - 2))(p + 2(n - 2)) * 2) \text{ (Bytes)}$
L3, cuando: CP < L2 size	$x * y * i * \left(\frac{3}{n} + \frac{2}{m} + \frac{2}{p} + \frac{4n}{mp}\right) \text{ (Bytes)}$
L2, cuando L1_size < CP2	$\left(\frac{xy}{mp}\right) * \frac{i}{n} ((m + 2n)(p + 2n) + 2(R y W) * (m)(p) + \sum_{x=1}^{n-1} [3(m + 2x)(p + 2x)]) \text{ (Bytes)}$
L2, cuando CP2 < L1_size < CP	$\left(\frac{xy}{mp}\right) * \frac{i}{n} ((m + 2n)(p + 2n) + 2 * (m)(p) +$

	$(m + 2(n - 1))(p + 2(n - 1)) * 2 + (m + 2(n - 2))(p + 2(n - 2)) * 2 \text{ (Bytes)}$
L2, cuando: CP < L1 size	$x * y * i * (\frac{3}{n} + \frac{2}{m} + \frac{2}{p} + \frac{4n}{mp}) \text{ (Bytes)}$
L1, cuando L1_size < CP2	$(\frac{xy}{mp}) * \frac{i}{n} ((m + 2n)(p + 2n) + 2(R y W) * (m)(p) + \sum_{x=1}^{n-1} [3(m + 2x)(p + 2x)]) \text{ (Bytes)}$
L1, cuando CP2 < L1_size < CP	$(\frac{xy}{mp}) * \frac{i}{n} ((m + 2n)(p + 2n) + 2(R y W) * (m)(p) + (m + 2(n - 1))(p + 2(n - 1)) + (m + 2(n - 2))(p + 2(n - 2)) + \sum_{x=1}^{n-1} [2(m + 2x)(p + 2x)]) \text{ (Bytes)}$
L1, cuando: CP < L1 size	$(\frac{xy}{mp}) * \frac{i}{n} ((m + 2n)(p + 2n) + 2(R y W) * (m)(p) + \sum_{x=1}^{n-1} [2(m + 2x)(p + 2x)]) \text{ (Bytes)}$

Analisis codigo assembler

parte interna

Código assembler de bucle interno de actualizar tablero (las pruebas anteriores fueron sin actualizar a una versión de gcc actual)

```
0,06 1fd: vmovdqu    (%r11,%rax,1),%ymm0
0,10  vmovdqu    0x0(%r13,%rax,1),%ymm1
2,12  vpaddb    (%rcx,%rax,1),%ymm0,%ymm0
2,50  vpaddb    (%r10,%rax,1),%ymm1,%ymm1
0,18  vpaddb    %ymm1,%ymm0,%ymm0
1,64  vmovdqu    (%r9,%rax,1),%ymm1
73,82 vpaddb    (%r8,%rax,1),%ymm1,%ymm1
1,01  vpaddb    %ymm1,%ymm0,%ymm0
5,51  vmovdqu    (%rbx,%rax,1),%ymm1
1,24  vpaddb    (%rdi,%rax,1),%ymm1,%ymm1
0,78  vpaddb    %ymm1,%ymm0,%ymm0
1,57  vpcmpeqb (%rdx,%rax,1),%ymm3,%ymm1
0,42  vpcmpeqb %ymm5,%ymm0,%ymm10
0,48  vpcmpeqb %ymm4,%ymm0,%ymm0
1,40  vpcmpeqb %ymm3,%ymm1,%ymm1
0,16  vpand     %ymm10,%ymm1,%ymm1
0,30  vpand     %ymm2,%ymm1,%ymm1
2,18  vpblendvb %ymm0,%ymm2,%ymm1,%ymm0
0,48  vmovdqu    %ymm0,(%rsi,%rax,1)
0,04  add       $0x20,%rax
0,02  cmp       %rax,0x70(%rsp)
1,90  jne       1fd
0,02  testb     $0x1f,0x3c(%rsp)
```

Vectorizadas == V (técnicamente sería SIMD), Instrucciones == Ins.

Instrucciones LOAD V.	9 (Ins.)
Instrucciones WRITE V.	1 (Ins.)
Instrucciones ADD V.	7 (Ins.)
Instrucciones CMP V.	4 (Ins.)
Instrucciones AND V.	2 (Ins.)
Instrucciones LEND V.	1 (Ins.)
Instrucciones ADD	1 (Ins.)
Instrucciones CMP	1 (Ins.)
Instrucciones JNE	1 (Ins.)
Total instrucciones	27

El ensamblador parece no optimizar el código, solo hace lo que se expone en la función y ya. Esto muestra que el compilador no ha podido encontrar alguna mejora al algoritmo.

Por tanto, ahora podemos proceder a deducir cuantas operaciones se realizan por elemento.

Partamos que por bloque hacemos :

$$\sum_{x=1}^{n-1} [(m + 2x)(p + 2x)] + 1 \quad (\text{operaciones/generar una celda destino})$$

Ahora bien, el total de operaciones se define por (y generalizando mucho y omitiendo que no todo se puede vectorizar si queda un elemento, etc):

$$\sum_{x=0}^{n-1} [(m + 2x)(p + 2x)] \quad (\text{operaciones/generar bloque destino})$$

Ahora, al estar vectorizado el código, cada iteración del bucle interno (omitiendo bordes) itera cada 32 elementos.

Por tanto, el total de iteraciones por bloque destino tiene este aspecto:

$$\left(\sum_{x=0}^{n-1} [(m + 2x)(p + 2x)] \right) / 32$$

Ahora, la ecuación general solo necesita multiplicar por la cantidad de bloques destino que vamos a computar, y ya tendríamos el total global:

$$\left(\frac{xy}{mp} \right) * \frac{i}{n*32} * \left(\sum_{x=0}^{n-1} [(m + 2x)(p + 2x)] \right)$$

Ahora solo falta saber los puertos de ejecución para multiplicar por los ciclos de cada iteración si la memoria no fuera un limitante y dividir por lo que tarda un ciclo en un segundo para obtener el tiempo en segundos.

<https://cdrdv2-public.intel.com/779559/355308-Software-Optimization-Manual-048-Changes-Doc-2.pdf>

Obtenemos de aquí en la página 15 en la figura Figure 2-1 la imagen de como son los puertos y en la página 15 obtenemos lo que buscamos en la Table 2-1:

Port 0	Port 1 ¹	Port 2	Port 3	Port 4	Port 5 ²	Port 6	Ports 7, 8	Port 9	Port 10	Port 11
INT ALU LEA INT Shift Jump1	INT ALU ³ LEA INT Mul INT Div	Load	Load	Store Data	INT ALU LEA INT MUL Hi	INT ALU LEA INT Shift Jump2	Store Address	Store Data	INT ALU LEA	Load
FMA Vec ALU Vec Shift FP Div	FMA* Fast Adder* Vec ALU* Vec Shift* Shuffle*				FMA** Fast Adder Vec ALU Shuffle					

Y en la Table 2-2 la imagen completa al final de la misma página y un poco de la posterior:

Execution Unit	# of Unit	Instructions
ALU	5 ²	add, and, cmp, or, test, xor, movzx, movsx, mov, (v)movdqu, (v)movdqa, (v)movap*, (v)movup*
SHFT	2 ³	sal, shl, rol, adc, sarx, adcx, adox, etc.
Slow Int	1	mul, imul, bsr, rcl, shld, mulx, pdep, etc.
BM	2	andn, bextr, blsi, blmsk, bzhi, etc.
Vec ALU	2x256-bit 1x512-bit	(v)add, (v)cmp, (v)max, (v)min, (v)sub, (v)cvtps2dq, (v)cvtdq2ps, (v)cvtsd2sl, (v)cvttss2sl
	3x256-bit 2x512-bit	(v)pand, (v)por, (v)pxor, (v)movq, (v)movq, (v)movap*, (v)movup*, (v)andp*, (v)orp*, (v)paddb/w/d/q, (v)blendv*, (v)blendp*, (v)pblendd
Vec_Shft	2x256-bit 1x512-bit	(v)psllv*, (v)psrlv*, vector shift count in imm8
VEC Add (in VEC FMA)	2x256-bit 1x512-bit	(v)add*, (v)cmp*, (v)max*, (v)min*, (v)sub*, (v)padds*, (v)paddus*, (v)psign, (v)pabs, (v)pavgb, (v)pcmpeq*, (v)pmax, (v)cvtps2dq, (v)cvtdq2ps, (v)cvtsd2si, (v)cvttss2si

Execution Unit	# of Unit	Instructions
VEC Fast Add	2x256-bit 1x512-bit	(v)add*, (v)addsub*, (v)sub*
Shuffle	2x256-bit 1x512-bit	(v)shuf*, vperm*, (v)pack*, (v)unpck*, (v)punpck*, (v)pshuf*, (v)pslldq, (v)alignr, (v)pmovzx*, vbroadcast*, (v)pslldq, (v)psrldq, (v)pblendw (new cross lane shuffle on both ports)
Vec Mul/FMA	2x256-bit (1 or 2)x512-bit	(v)mul*, (v)pmul*, (v)pmadd*
SIMD Misc	1	STTNI, (v)pclmulqdq, (v)psadw, vector shift count in xmm
FP Mov	1	(v)movsd/ss, (v)movd gpr
DIVIDE	1	divp*, divs*, vdiv*, sqrt*, vsqrt*, rcp*, vrcp*, rsqrt*, idiv

Obtenemos de la siguiente línea en la página 14 la cantidad de retriive del procesador:
Wider allocation (5 -> 6 uops per cycle) and retirement (4 -> 8 uops per cycle) width.

Obviamente, solo nos interesan comparar los LOAD, ADD, CMP, casos donde muchas instrucciones combinadas que usan los mismos puertos y retrieve (vectorizadas las instrucciones) pues será lo que muy seguramente nos limite pues son las que poseen mayor peso en la parte interna del bucle. Por tanto, después de analizar las tablas podemos al fin saber la latencia por iteración:

Instrucciones LOAD V.	$9 / (P2+P3+P11) = 3$ ciclos / iteración
Instrucciones ADD V.	$7 / (P0+P1+P5) = 2.33$ ciclos / iteración
Instrucciones CMP V.	$4 / (P0+P1+P5) = 1.33$ ciclos / iteración
7 ADDs + 4 CMPs + 2 ANDs + 1 LEND	$(7+4+2+1) / (P0+P1+P5) = 4.67$ ciclos / iteración
Total instrucciones	$27 / 8 = 3.375$ ciclos / iteración

Total, que el limitante si fuera cómputo sería culpa de las instrucciones vectoriales de los puertos P0, P1 y P5. La fórmula ahora queda así :

$$\left(\frac{xy}{mp}\right) * \frac{i}{n*32} * \left(\sum_{x=0}^{n-1} [(m + 2x)(p + 2x)]\right) * 4.67 \text{ (ciclos)}$$

Ahora, dividimos entre la frecuencia del procesador, debido a que esta es variable supondremos un caso de overclocking, que consiste en llevar al límite el procesador, por tanto partimos de la siguiente frecuencia 4400 MHz. Lo pasamos a Hz -> $4.4*10^9$ Hz. Ahora sacamos la fórmula final:

$$\left(\frac{xy}{mp}\right) * \frac{i}{n*32} * \left(\sum_{x=0}^{n-1} [(m + 2x)(p + 2x)]\right) * \frac{(4.67/6)}{4.4*10^9} \text{ (s)}$$

Sino fuera que usamos multicore, que debido a ello y siendo muy optimistas cada iteración no serán 4.67 ciclos, sino 4.67/6 ciclos, puesto que tenemos 6 cores

Estimar el BW de las caches

Vale, hasta ahora hemos planteado todas las fórmulas de memoria, analizado el ensamblador y poco más. Ahora lo que nos falta son dos cosas. Primero de todo hay que conseguir poner en función del tiempo las fórmulas. Segundo hay que hacer una ecuación para, pensando que la capacidad de cómputo no es limitante, estimar el tiempo que tardaría. Y por otro lado si la memoria no fuera un problema, cuánto tardaría si fuera por cómputo. La idea es encontrar el punto en que la función max(tiempo limitado por memoria, tiempo limitado por cómputo) sea mínima.

Por tanto, extraemos el bandwidth de las caches del procesador vía un código rápido :

<https://github.com/Kobzol/hardware-effects/blob/master/cache-hierarchy-bandwidth/cache-hierarchy-bandwidth.cpp> que proviene de una de las fuentes que nos otorga la asignatura. Lo modificamos un poco para hacer múltiples pruebas .

```
int main(int argc, char** argv)
{
    if (argc < 2)
    {
        std::cout << "Usage: cache-hierarchy-bandwidth <size>" <<
std::endl;
        return 1;
    }
    for (int i = 1; i < argc; i++)
    {
        size_t count = std::stoll(argv[i]) / sizeof(Type);
        std::vector<Type> data(count, 0);
        test_memory(data);
    }

    return 0;
}
```

Este programa lo ejecutamos así :

[aa-26@aolin-login testBandwidthCache]\$ make test

g++ -O3 -march=native main.cpp -o benchmark

perf stat ./benchmark 1000 2000 3000 4000 5000 6000 7000 8000 9000 10000 20000
30000 40000 50000 60000 70000 80000 90000 100000 200000 300000 400000 500000
600000 700000 800000 900000 1000000 2000000 3000000 4000000 5000000 6000000
7000000 8000000 9000000 10000000 20000000 30000000 40000000

Miramos el tamaño de las caches con lscpu y obtenemos el siguiente resultado:

Para L1 en promedio da un rango de entre **110-123 GB/s por core**

Para L2 en promedio no da un rango de entre **58-61 GB/s por core**

Para L3 haría falta hacer el programa multicore y cambiarlo un poco:

```
#pragma omp parallel
for (int i = 0; i < REPETITIONS; i++)
{
    #pragma omp for
    for (size_t j = 0; j < size; j++)
    {
        memory[j] = value;
    }
}
```

Se ejecuta con :

```
[aa-26@aolin-login testBandwidthCache]$ make test
g++ -O3 -march=native -fopenmp main.cpp -o benchmark
perf stat ./benchmark 1310720 1400000 1500000 1600000 1700000 1800000
1900000 2000000 10000000 11000000 12000000 13000000 14000000 15000000
16000000 17000000 18874368 27000000 37000000 47000000 57000000
Bytes Moved (GiB/s) 315.847 Size i bytes:1310720
Bytes Moved (GiB/s) 318.528 Size i bytes:1400000
Bytes Moved (GiB/s) 286.046 Size i bytes:1500000
Bytes Moved (GiB/s) 279.657 Size i bytes:1600000
Bytes Moved (GiB/s) 334.224 Size i bytes:1700000
Bytes Moved (GiB/s) 346.053 Size i bytes:1800000
Bytes Moved (GiB/s) 332.688 Size i bytes:1900000
Bytes Moved (GiB/s) 332.243 Size i bytes:2000000
Bytes Moved (GiB/s) 239.985 Size i bytes:10000000
Bytes Moved (GiB/s) 228.66 Size i bytes:11000000
Bytes Moved (GiB/s) 204.554 Size i bytes:12000000
Bytes Moved (GiB/s) 203.501 Size i bytes:13000000
Bytes Moved (GiB/s) 198.735 Size i bytes:14000000
Bytes Moved (GiB/s) 194.579 Size i bytes:15000000
Bytes Moved (GiB/s) 214.654 Size i bytes:16000000
Bytes Moved (GiB/s) 211.629 Size i bytes:17000000
Bytes Moved (GiB/s) 217.094 Size i bytes:18874368
Bytes Moved (GiB/s) 55.143 Size i bytes:27000000
Bytes Moved (GiB/s) 27.3563 Size i bytes:37000000
Bytes Moved (GiB/s) 21.6738 Size i bytes:47000000
Bytes Moved (GiB/s) 24.8554 Size i bytes:57000000
```

Como se puede observar primero nuestra observación sobre la RAM era cierta, prácticamente da ese 30,3 GB/s (las variaciones pueden ser de multitud de motivos pero el parecido está).

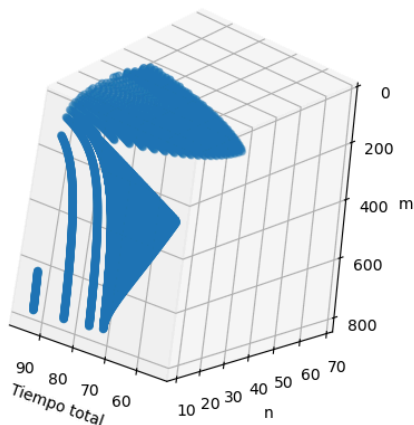
Y el BW agregado para L3 para todos los cores nos da entre 194 GB/s a 217 GB/s. La verdad que al hacer diversas pruebas el número al principio es demasiado grande y luego se estabiliza, así que tomamos los “peores casos” por así decirlo. Pero lo que nos da realmente es un rango ridículo de 194 GB/s a 346 GB/s. La verdad que difícil de creer. Por ello tomamos estos peores valores como referencia. En fin, nos queda que en L3 por core tenemos cogiendo de referencia ese 217, 36.16 GB/s por core.

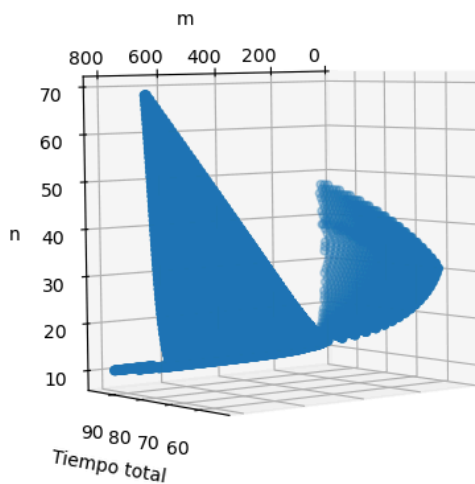
	Agregado o total (GB/s)	Por core (GB/s)
L1d	660-738	110-123
L2	348-366	58-61
L3	194-217	32.3-36.16

Montar y analizar la función del tiempo límite del código

Ahora nos enfrentamos a una ecuación por partes, tenemos diversas opciones. Una podría solucionarlo con cálculos, otros lo dejarían así y ya está. Nuestra solución ha sido un poco alternativa, hemos hecho uso de python y nos ha salido esta gráfica:

Siendo $m=p$ porque sino no podemos graficarla.

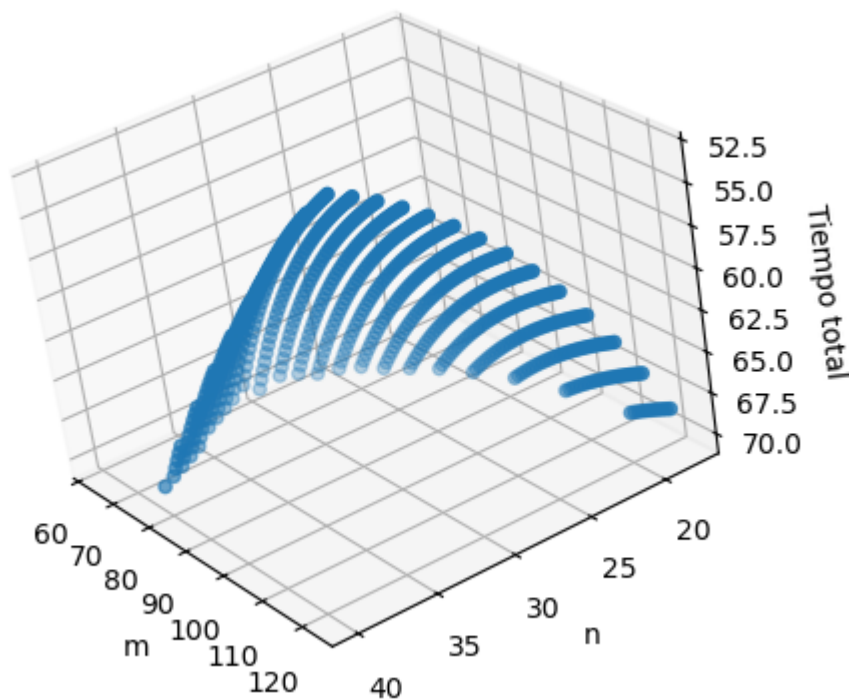




Modificamos el script el `limiteTilingBestCase.py`:

<https://github.com/jaimeamse/Game-of-Life-optimization/blob/main/limiteTilingBestCase.py>

Ahora, vamos a probar con una n máxima de 1000 y vamos a buscar solo el punto mínimo. Mostramos sólo los puntos que el tiempo total sea < 70 s.



```
m del minimo:97
n del minimo:31
Tiempo minimo:53.33645112344446
```

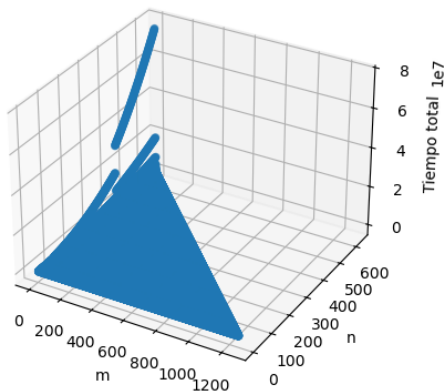
Corremos otra vez los cálculos añadiendo estas líneas:

```
m = x_vals[posMinTime]
n = z_vals[posMinTime]
```

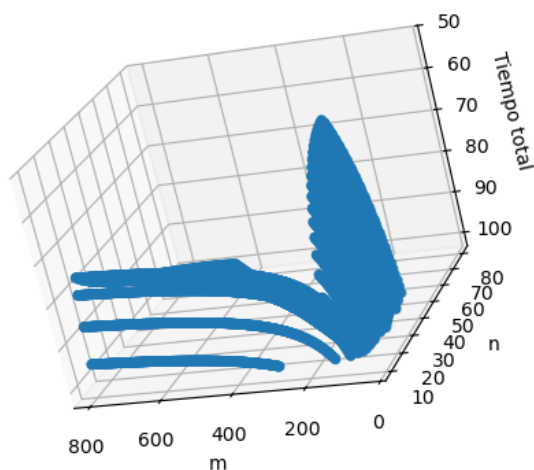
```
p = m
print("Tiempos individuales para cada nivel de memoria:")
print([
    getTimeL1(m, p, n),
    getTimeL2(m, p, n),
    getTimeL3(m, p, n),
    getTimeRAM(m, p, n)
])
```

```
Tiempos individuales para cada nivel de memoria:
[52.885710057238526, 12.978050643576296, 8.247904812903784, 53.33645112344446]
```

Ahora, habiendo cierta curiosidad, si no limitamos los puntos < 70s ocurre esto:



Ahora, si limitamos a < 100s



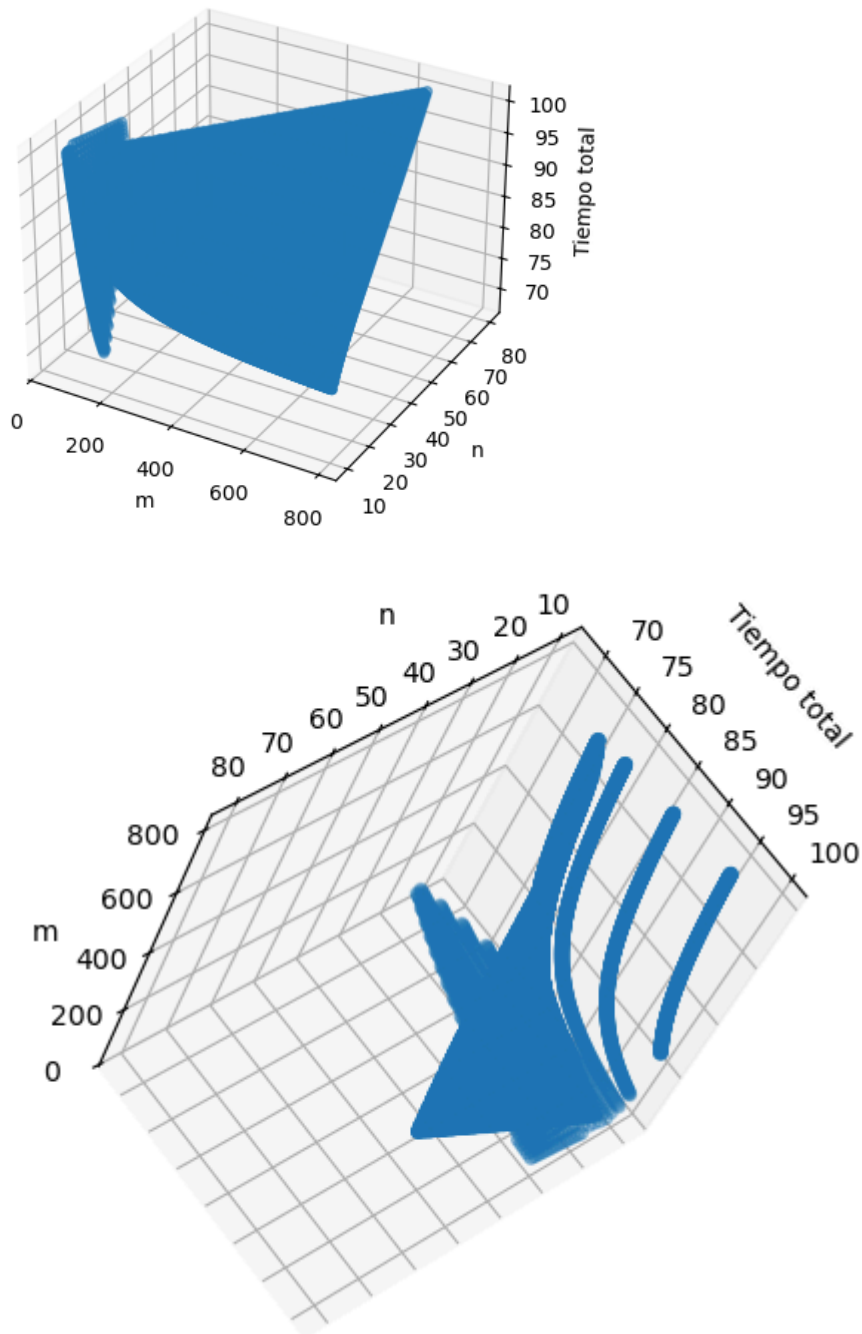
Como se puede observar la función tiene formas extrañas, pero bueno, nos sale ya un buen resultado. Ahora nos falta analizar cuando tenemos en cuenta las limitaciones de computo con esta fórmula que hemos deducido antes:

$$\left(\frac{xy}{mp}\right) * \frac{i}{n^{32}} * \left(\sum_{x=0}^{n-1} [(m + 2x)(p + 2x)]\right) * \frac{(4.67/6)}{4.4 \cdot 10^9} \text{ (s)}$$

Modificamos el script el `limiteTilingBestCase.py`:

<https://github.com/jaimeamse/Game-of-Life-optimization/blob/main/limiteTilingBestCase.py>

Obtenemos el resultado:



Ahora vamos a desvelar las mejores condiciones:

```
m del minimo:123
n del minimo:18
Tiempo minimo:68.3610099220364
Tiempos individuales para cada nivel de memoria:
[40.56282250240629, 15.02480060002884, 10.571290194129341, 68.3610099220364]
Tiempo por computo teorico:
67.00563069707697
```

Como se puede observar por el output de la terminal, el código estará limitado por el BW de la RAM del ordenador, aunque por poco es limitado por el cómputo. Por tanto sería prudente saber que aunque mejoraremos la RAM y pese a ser el cuello de botella solo mejoraremos 1.3 s que resulta en un mero 1.02x. Por tanto el código sigue siendo memory-bandwidth bound pero por poco. La mejor forma de mejorar este rendimiento sería proceder con un procesador con muchos más cores y mejorar a la vez la RAM para una mejora significativa.

La última capa de dificultad que añadiremos a la fórmula y que asumimos para que sea correcta

Primero de todo, como hemos visto la fórmula tiene diferentes limitaciones y cosas que no tiene en cuenta. Primero, $n > 3$, segundo, piensa que todos los accesos son byte a byte cuando si son desalineados cojemos todo un bloque de caché al principio o al final. Asumimos que las tareas se asignan como una cola de tareas y que los bloques si o si al inicio los tendrán que pedir de memoria, no que queden en caché (cosa que podría mejorar). Por tanto, teniendo esto en cuenta, sabemos que podemos cambiar un poco las ecuaciones para superar estas limitaciones, procederemos a hacerlo y mostraremos el peor resultado y el mejor resultado posibles.

El tamaño de bloque es 64 bits debido que lo obtenemos de hacer en la terminal :

```
getconf LEVEL1_DCACHE_LINESIZE
```

Para saber el rango de tiempo lo que haremos pues es suponer el peor y el mejor caso. Y sabremos que el valor de tiempo tendrá que estar entre estos valores.

Uso de recursos de	Fórmula de bytes transferidos
Memoria Principal (DDR4) CP*NT < L3_size < CP*NT (L3 -> solo le entran los 2 bloques personales)	$\left(\frac{(x+7)y}{(m+14)p}\right) * \frac{i}{n} ((m + 2n + 14)(p + 2n) + 2 * (m + 14)(p) + (m + 2(n - 1) + 14)(p + 2(n - 1)) * 2 + (m + n - 2 + 14)(p + n - 2) * 2) \text{ (Bytes)}$
Memoria Principal (DDR4) CP*NT < L3 size	$\left(\frac{(x+7)y}{(m+14)p}\right) * \frac{i}{n} ((m + 2n + 14)(p + 2n) + 2 * (m + 14)(p)) \text{ (Bytes)}$
L3, cuando L2_size < CP2 y CP2*NT < L3_size (Los arrays personales solo entran en L3)	$\left(\frac{(x+7)y}{(m+14)p}\right) * \frac{i}{n} ((m + 2n + 14)(p + 2n) + 2 * (m + 14)(p) + \sum_{x=1}^{n-1} [3(m + 2x + 14)(p + 2x)]) \text{ (Bytes)}$
L3, cuando CP2 < L2_size < CP	$\left(\frac{(x+7)y}{(m+14)p}\right) * \frac{i}{n} ((m + 2n + 14)(p + 2n) + 2 * (m + 14)(p) + (m + 2(n - 1) + 14)(p + 2(n - 1)) * 2 + (m + n - 2 + 14)(p + n - 2) * 2) \text{ (Bytes)}$
L3, cuando: CP < L2 size	$\left(\frac{(x+7)y}{(m+14)p}\right) * \frac{i}{n} ((m + 2n + 14)(p + 2n) + 2 * (m + 14)(p)) \text{ (Bytes)}$
L2, cuando L1_size < CP2	$\left(\frac{(x+7)y}{(m+14)p}\right) * \frac{i}{n} ((m + 2n + 14)(p + 2n) + 2 * (m + 14)(p) + \sum_{x=1}^{n-1} [3(m + 2x + 14)(p + 2x)]) \text{ (Bytes)}$

L2, cuando CP2 < L1_size < CP	$\left(\frac{(x+7)y}{(m+14)p}\right) * \frac{i}{n} ((m + 2n + 14)(p + 2n) + 2 * (m + 14)(p) + (m + 2(n - 1) + 14)(p + 2(n - 1)) * 2 + (m + n - 2 + 14)(p + n - 2) * 2) \text{ (Bytes)}$
L2, cuando: CP < L1 size	$\left(\frac{(x+7)y}{(m+14)p}\right) * \frac{i}{n} ((m + 2n + 14)(p + 2n) + 2 * (m + 14)(p)) \text{ (Bytes)}$
L1, cuando L1_size < CP2	$\left(\frac{(x+7)y}{(m+14)p}\right) * \frac{i}{n} ((m + 2n + 14)(p + 2n) + 2 * (m + 14)(p) + \sum_{x=1}^{n-1} [3(m + 2x + 14)(p + 2x)]) \text{ (Bytes)}$
L1, cuando CP2 < L1_size < CP	$\left(\frac{(x+7)y}{(m+14)p}\right) * \frac{i}{n} ((m + 2n + 14)(p + 2n) + 2 * (m + 14)(p) + (m + 2(n - 1) + 14)(p + 2(n - 1)) + (m + 2(n - 2) + 14)(p + 2(n - 2)) + \sum_{x=1}^{n-1} [2(m + 2x + 14)(p + 2x)]) \text{ (Bytes)}$
L1, cuando: CP < L1 size	$\left(\frac{(x+7)y}{(m+14)p}\right) * \frac{i}{n} ((m + 2n + 14)(p + 2n) + 2 * (m + 14)(p) + \sum_{x=1}^{n-1} [2(m + 2x + 14)(p + 2x)]) \text{ (Bytes)}$

Andreu Elias Puigvert Gómez - 1702901
Jaime Amselem Herrero - 1704539

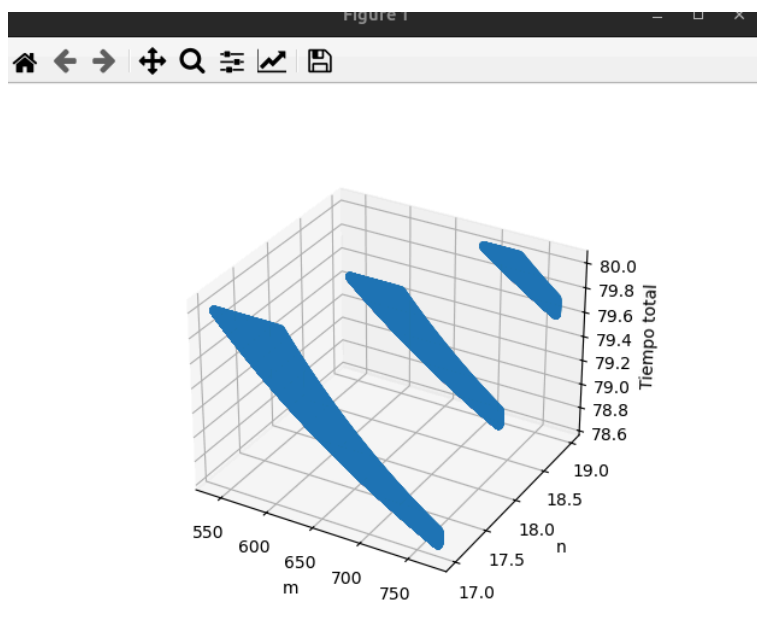
Procedemos a computar todos los casos cuando m y $p < 1000$:

Usando este script:

<https://github.com/jaimeamse/Game-of-Life-optimization/blob/main/limiteTilingWorstCase.py>

```
m del minimo:765
p del minimo:778
n del minimo:17
Tiempo minimo:78.66026033655565
Tiempos individuales para cada nivel de memoria:
[41.475599233155734, 78.66061923529536, 20.938471704547222, 77.00195077807342]
Tiempo por computo teorico:
54.07790949751714
□
```

Cuando $t < 80$ s.



Optimizaciones del programa y resultados parte 2.

Versión Optimizaciones por columnas parte 2

Posibles vías de mejora, parte 2

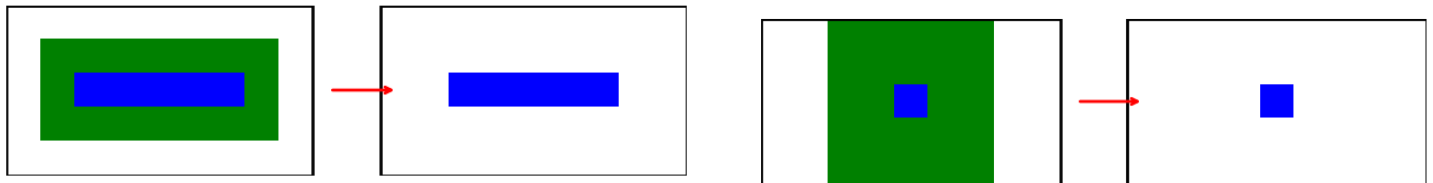
Resumen:

	Possible speed up para 100000x100000 it = 100 si siempre memoria DDR5 el límite	Dificultad de vectorizar(0-10)	Dificultad de implementar
Compresión bitmap	8x (si sigue limitando el cómputo)	10	10
Tiling	1,18x-1.38x	0	9
Dos iteraciones en una	2x	5	7,5

Como hemos podido calcular, la mejora esperada no es muy prometedora para la dificultad que ofrece la mejora de tiling, compresión a bitmap la intentamos y digamos que no salió muy bien. Por ende, nos queda sólo dos iteraciones en una para intentar mejorar.

Hacer dos iteraciones en una

Cómo funciona



Como vemos en las imágenes, si queremos actualizar la celda directamente dos iteraciones a futuro, necesitamos tomar más vecinos convirtiendo el problema en un stencil-9 a stencil-25. Debido a las anteriores optimizaciones accederemos mucho menos a memoria principal, pero también aumentaremos el tiempo de cómputo debido a que repetimos múltiples veces los mismos cálculos.

El código que ahora se encuentra disponible en el git:

<https://github.com/jaimeamse/Game-of-Life-optimization/blob/main/GOL.c>

Entonces como decíamos, para medir cuánto de menos datos transfiere de M.P nuestro código tenemos esta fórmula (asumiendo siempre accesos alineados y que $i \geq 2$, y que es el peor caso, es decir hay que hacer dos veces como el original pues nos falta modificar el la generación del checksum y no tenemos en cuenta los bordes):

$$2 * 3 * x * y + \frac{i-2}{2} (x * y * 3)$$

Cuando antes era esta fórmula:

$$3 * x * y * i$$

Resumen:

En el límite al infinito es un 2x, pero por lo general será un 1.9x de menos transferencia de datos.

Aparte de ello, hemos tenido en cuenta en la optimización que el resultado de bien y que se gestionen correctamente los bordes de manera que sea correcto el resultado.

Las dificultades de implementación fueron que no hiciera llamadas como subrutinas a evaluar celda y que no hiciera saltos condicionales. La primera dificultad la superamos usando inline al declarar la función.

La segunda resulta que el `||` causaba este salto condicional puesto que el compilador pensaba que no hacía falta evaluar las condiciones derechas si las de izquierda eran ciertas causando que no vectorizara. Para solucionarlo cambiamos los operadores de `||` a `|` en

Andreu Elias Puigvert Gómez - 1702901
Jaime Amselem Herrero - 1704539

evaluar_celda_2_step y && a & por un motivo más lógico pues queríamos una operación and no una comparación, pero no debería afectar.

Aparte de esto, al hacer un análisis con nuestra herramienta hemos descubierto que si ponemos 3500 columnas por tarea + 12 threads obtenemos mejores resultados.

Resultados

El resultado pues es el siguiente (en gcc más moderno disponible):

```
./GOL.c:351:17: optimized: loop vectorized using 32 byte vectors
perf stat ./GOL
Matrix is 100000 x 100000.
Number of iterations: 100
Checksum = 739936371

Performance counter stats for './GOL':

    582.700,93 msec task-clock:u          #    7,504 CPUs utilized
         0      context-switches:u       #    0,000 /sec
         0      cpu-migrations:u         #    0,000 /sec
    4.701.201    page-faults:u           #    8,068 K/sec
  2.297.812.435.656 cpu_core/cycles:u/    #    3,943 G/sec
  2.896.503.721.897 cpu_core/instructions:u/ #    4,971 G/sec
   32.786.941.142 cpu_core/branches:u/   #   56,267 M/sec
    507.274.056  cpu_core/branch-misses:u/ #   870,556 K/sec

    77,649865007 seconds time elapsed

    574,624119000 seconds user
     5,661069000 seconds sys
```

Es un speedup de la versión anterior de 1.19x (en gcc 8.X.X contra en gcc más moderno disponible, nos dimos cuenta tarde).

```
Checksum = 739936371

Performance counter stats for './GOL':

    591.692,29 msec task-clock:u          #    8,793 CPUs utilized
         0      context-switches:u       #    0,000 /sec
         0      cpu-migrations:u         #    0,000 /sec
    3.568.713    page-faults:u           #    6,031 K/sec
  2.337.105.566.421 cpu_core/cycles:u/    #    3,950 G/sec
  3.095.462.485.807 cpu_core/instructions:u/ #    5,232 G/sec
   43.808.559.130 cpu_core/branches:u/   #   74,039 M/sec
    1.152.987.973  cpu_core/branch-misses:u/ #    1,949 M/sec

    67,294241259 seconds time elapsed

    584,163268000 seconds user
     4,863561000 seconds sys

[aa-26@aolin-login proyecto]$
```

Es un speedup de la versión anterior de 1.36x (en gcc 8.X.X contra en gcc más moderno disponible, nos dimos cuenta tarde).

Datos de rendimiento en tiempo (resumen, compilador versión reciente):

X	Y	Iteraciones	Tiempo Código Columnas	SpeedUp	Hyperthreading para mejor caso
150	150	100000	0,63	1,12x	No
750	750	10000	0,39	0,56x	No
3000	3000	1000	0,34	0,63x	No
3100	3100	1000	0,40	0,64x	No
10000	10000	100	1,02	1,47x	No
20000	20000	100	3,65	1,41x	No
20000	100000	100	17,66	1,38x	No
100000	100000	100	84,36	1,24x	Sí

Datos globales del código directo (compilador versión reciente):

X	Y	Iteraciones	Ciclos (G)	Instrucciones (G)	IPC	Tiempo	Reloj	(t)/(it*ciclo)	N_threads
150,00	150,00	100000,00	4,29	17,42	4,07	0,99	4,32	0,44	1,00
750,00	750,00	10000,00	7,60	25,73	3,38	1,75	4,34	0,31	1,00
3000,00	3000,00	1000,00	10,39	27,42	2,64	2,40	4,33	0,27	1,00
3100,00	3100,00	1000,00	11,01	30,23	2,74	2,54	4,34	0,26	1,00
10000,00	10000,00	100,00	11,53	29,23	2,54	2,66	4,33	0,27	1,00
20000,00	20000,00	100,00	46,70	116,84	2,50	10,76	4,34	0,27	1,00
20000,00	100000,00	100,00	233,76	583,79	2,50	53,72	4,35	0,27	1,00
100000,00	100000,00	100,00	1189,26	2894,20	2,43	274,45	4,33	0,27	1,00
150,00	150,00	100000,00	13,30	34,40	2,59	0,56	3,97	0,25	6,00
750,00	750,00	10000,00	16,59	69,39	4,18	0,70	3,98	0,12	6,00
3000,00	3000,00	1000,00	12,61	32,06	2,54	0,54	3,97	0,06	6,00
3100,00	3100,00	1000,00	14,67	34,36	2,34	0,63	3,98	0,07	6,00
10000,00	10000,00	100,00	14,19	29,45	2,08	0,70	3,96	0,07	6,00

20000,00	20000,00	100,00	51,81	117,58	2,27	2,58	3,96	0,06	6,00
20000,00	100000,00	100,00	257,32	587,62	2,28	12,83	3,97	0,06	6,00
100000,00	100000,00	100,00	1257,57	2894,64	2,30	74,34	3,97	0,07	6,00
150,00	150,00	100000,00	35,85	44,12	1,23	0,75	3,98	0,33	12,00
750,00	750,00	10000,00	64,09	134,16	2,09	1,34	3,98	0,24	12,00
3000,00	3000,00	1000,00	37,16	58,42	1,57	0,79	3,98	0,09	12,00
3100,00	3100,00	1000,00	30,96	35,04	1,13	0,66	3,97	0,07	12,00
10000,00	10000,00	100,00	27,89	29,44	1,06	0,70	3,96	0,07	12,00
20000,00	20000,00	100,00	109,18	117,99	1,08	2,76	3,96	0,07	12,00
20000,00	100000,00	100,00	579,35	589,34	1,02	14,72	3,97	0,07	12,00
100000,00	100000,00	100,00	2347,23	3095,51	1,32	68,03	3,95	0,07	12,00

Limitaciones

Si hacemos una prueba con un thread, matriz 200x200 y iteraciones 100000 haciendo los cálculos básicos:

Siendo los ciclos totales del programa = 7114863825

Siendo el total de iteraciones = $100000 \times 200 \times 200 / 32 = 125000000$

$7114863825 \div (125000000) = 56,91$ ciclos / iteración aproximadamente

Por ende si hacemos cálculos con el caso 100000×100000 , iteraciones = 100:

Total iteraciones = $100000 \times 100000 \times 100 / 32 = 3125000000$

Total ciclos = $3125000000 \times 56,91 = 1778,4375$ G ciclos

Tomando de referencia que ejecutando aplicaciones multicores el reloj tiene una frecuencia de 3,961 G ciclos / s

Tiempo estimado si limitado por cómputo = $(1778,4375 \div 6 \text{ cores}) / 3,961 = 74,83$ s

Sabiendo aún que el 56,91 ciclos / iteración posiblemente esté por encima del valor real, pese a ello parece ser que realmente está limitado por el BW de cómputo en esta ocasión el código.

Por ende, es por ello que el código pese nosotros esperar un 1.9x obtenemos un respetable 1.24x en el caso más grande que hemos probado.

Posibles vías de mejora

Intentar combinar los bitmaps, tiling y iterar dos veces de una buscando la combinación correcta podría ser una vía. Otra forma sería intentar hacer bitmaps ya que prometen mucho rendimiento partiendo de dividir columnas. Realmente aún se pueden hacer muchas optimizaciones, el problema como siempre, es nuestro tiempo.

Conclusiones

Las conclusiones de este proyecto son diversas:

La presentación es un factor fundamental. Si no se dedica el tiempo suficiente a exponer el trabajo realizado, este puede perder relevancia, incluso aunque se le haya dedicado un gran esfuerzo. Saber expresar ideas complejas de forma clara y directa es clave: es necesario omitir la información irrelevante y centrarse en aquello que realmente aporta valor e interés.

La cooperación resulta esencial. Aunque no se mencionara inicialmente en las conclusiones, trabajar con un compañero que debe avanzar al mismo ritmo obliga a tener las ideas claras y bien estructuradas en un documento. Esto implica buscar la mejor manera de explicar conceptos que, de por sí, ya son difíciles de comprender. Se trata de un reto distinto y exigente, pero a cambio el conocimiento propio se amplía y se refina. Es un proceso bidireccional.

Mantener la documentación actualizada permite una mayor organización. El hecho de desarrollar el documento entregado en PDF de forma paralela al desarrollo de la aplicación ha facilitado estructurar, de manera más o menos correcta, la gran cantidad de información necesaria para llevar a cabo este proyecto.

Enlaces de recursos propios

Google sheets para las tablas nuevas y las de presentación:

https://docs.google.com/spreadsheets/d/19obqrpNW9OFioGuNY9P__RenLfJUyo1-cVKNgMWz8ns/edit?usp=sharing

Google sheets que usamos para ir desarrollando las mejoras:

<https://docs.google.com/spreadsheets/d/14I9baOuq75kteWTxfz15szRGh5aAH4kNgdUoHUsB52A/edit?usp=sharing>

Presentación que se entrega en conjunto este documento.

Github con el código subido: <https://github.com/jaimeamse/Game-of-Life-optimization>